

摘 要

当前大模型部署的前沿方向——多智能体系统中，多个智能体高频并发调用后端模型，形成大批量与长上下文叠加的复杂推理负载。这种负载在系统底层引发频繁的显存读写和访存瓶颈，使得大量推理算子性能受限。现有框架（如 Triton）开发便捷但优化表达能力受限，手写 CUDA 能逼近极限但开发成本过高。如何快速开发定制化、高性能的推理算子成为大模型高效部署的关键问题。

本文采用 TileLang 开发了一套优化的推理算子库，以直接应对上述性能瓶颈。针对 Qwen3-32B，本文系统识别并优化了六类核心算子。对于访存受限算子（RMSNorm、RoPE、SwiGLU、GQA Decode），通过算子融合与内存布局优化削减 HBM 流量；对于计算受限算子（GQA Prefill），通过改进的张量分块提升 Tensor Core 利用率；对于量化算子（W4A16 GEMV），融合 INT4 反量化与矩阵乘法降低访存开销。此外，本文设计了无侵入式的运行时算子替换机制，使优化算子能灵活集成到现有推理框架中，成功构建了支持多 Agent 并发推理的高吞吐原型系统。

NVIDIA A100 平台评测表明，相比标准 Triton 基线，TileLang 实现的算子取得显著加速，其中 RMSNorm、RoPE、GQA 算子性能提升 1.2 至 3.1 倍，SwiGLU 基本触及可优化上限，W4A16 GEMV 性能提升 5 至 8 倍。端到端测试显示，批大小为 4 时系统整体吞吐量提升 1.5 至 1.6 倍，批大小为 8 时提升 1.8 至 2.0 倍，墙上时间缩短 30% 至 50%。本方案证实能以低开发门槛获得显著性能收益，为大模型高性能算子定制化开发提供了实用参考。

关键词：大语言模型推理；多智能体系统；TileLang；算子优化；模型量化

Abstract

In multi-agent systems, a frontier direction for large language model deployment, multiple agents concurrently invoke backend models at high frequency, creating complex inference workloads that combine large batch sizes with long contexts. This workload causes frequent GPU memory read/write operations and memory-access bottlenecks at the system level, leaving numerous inference operators performance-limited. Existing frameworks such as Triton offer convenient development but limited expressiveness for optimization, while handwritten CUDA can approach hardware limits but incurs prohibitive development costs. How to rapidly develop customized, high-performance inference operators has become a key challenge for efficient large model deployment.

This thesis develops a set of optimized inference operators using TileLang to directly address the aforementioned performance bottlenecks. Targeting Qwen3-32B, we systematically identify and optimize six categories of core operators. For memory-bound operators (RMSNorm, RoPE, SwiGLU, GQA Decode), we reduce HBM traffic through operator fusion and memory layout optimization; for compute-bound operators (GQA Prefill), we improve Tensor Core utilization via enhanced tensor tiling; for quantized operators (W4A16 GEMV), we fuse INT4 dequantization with matrix multiplication to reduce memory access overhead. Additionally, we design a non-intrusive runtime operator replacement mechanism that enables optimized operators to integrate flexibly into existing inference frameworks, successfully constructing a high-throughput prototype system supporting concurrent multi-agent inference.

Evaluations on the NVIDIA A100 platform demonstrate significant speedups compared to standard Triton baselines: RMSNorm, RoPE, and GQA operators achieve 1.2 to 3.1 times speedup; SwiGLU essentially reaches the optimization ceiling; and W4A16 GEMV achieves 5 to 8 times speedup. End-to-end tests show 1.5 to 1.6 times overall throughput improvement at batch size 4 and 1.8 to 2.0 times improvement at batch size 8, with wall-clock time reduced by 30% to 50%. This approach confirms

that substantial performance gains can be achieved at low development cost, providing a practical reference for customized development of high-performance operators for large language models.

Keywords: Large Language Model Inference; Multi-Agent Systems; TileLang; Operator Optimization; Model Quantization

目 录

摘 要	I
Abstract	II
1 绪论	1
1.1 研究背景与问题	1
1.2 国内外研究现状	4
1.3 本文主要工作	6
1.4 论文结构安排	6
2 相关技术与基础	7
2.1 多 Agent 推理场景	7
2.2 GPU 体系结构与并行计算模型	8
2.3 Qwen3 推理中的热点算子	10
2.4 W4A16 权重量化技术	14
2.5 TileLang 编程模型	16
2.6 本章小结	18
3 系统设计与实现	19
3.1 系统总体架构	19
3.2 算子优化实现	21
3.3 推理调度引擎	38
3.4 交互展现层设计	40
3.5 本章小结	43
4 实验与评估	44
4.1 实验环境与方法论	44
4.2 算子微基准评测	45
4.3 端到端系统评估	54
4.4 消融实验	55
4.5 交互展现层功能验证	57

4.6 本章小结.....	59
5 总结与展望.....	60
5.1 主要工作总结.....	60
5.2 展望.....	60
5.3 本章小结.....	61
参考文献	62
致 谢	67

1 绪论

1.1 研究背景与问题

1.1.1 多 Agent 推理场景与负载特征

以 Claude Code^[1]、OpenClaw^[2] 为代表的多智能体（multi-agent，下文简称多 Agent）编程系统正成为大语言模型最主要的部署形态之一^[3]。在这类系统中，多个专用子智能体（Agent，如代码审查、缺陷检测、测试生成等）并发调用同一后端模型，各自携带数千甚至数万行的代码仓库片段或长文档作为上下文。相较于传统单轮对话，这一范式在推理系统上对硬件资源产生更大的压力，主要体现在三个方面。

第一，批量压力。多路并发使推理侧有效批量远大于传统单会话，解码（Decode）阶段权重加载被多路请求分摊，高带宽显存（High Bandwidth Memory, HBM）带宽因而常成为主导瓶颈^[4]。

第二，上下文压力。代码与文档片段使提示（prompt）上下文长度可达千词元乃至百万词元（token）量级，键值缓存（Key-Value Cache, KV Cache）在 Decode 阶段持续累积，单一对话可占用数十 GB 显存。大批量与长上下文两类压力相互叠加，急剧放大了对 HBM 容量与带宽的需求。

第三，容量竞争压力。大参数模型的半精度浮点（16-bit Floating Point, FP16）权重本身已占用大量显存，多卡部署下每张卡可用于 KV Cache 的剩余空间有限，权重与 KV Cache 争抢有限显存的矛盾随并发数与上下文增长只会愈发突出。

1.1.2 推理算子的两类性能瓶颈

Transformer（基于自注意力机制的变换器）推理链路中的算子，依其算术强度（浮点运算量与 HBM 访存字节数之比，FLOP/Byte）可分为两类。

计算受限（compute-bound）算子（如长序列预填充（Prefill）阶段的分组查询注意力（Grouped Query Attention, GQA）矩阵乘）具有高算术强度，性能上限由张量核心（Tensor Core）算力决定，此时 HBM 带宽已不是主要约束。

访存受限（memory-bound）算子则相反——均方根层归一化（Root Mean

Square Layer Normalization, RMSNorm)、旋转位置编码 (Rotary Position Embedding, RoPE)、Swish 门控线性单元 (Swish-Gated Linear Unit, SwiGLU)、GQA 解码 (Decode) 阶段注意力及多层感知机 (Multi-Layer Perceptron, MLP) 矩阵-向量乘 (Matrix-Vector Multiply, GEMV) 等算子的算术强度均远低于 A100 的屋脊点 (≈ 156 FLOP/Byte), 其性能上限由 HBM 带宽 (2.0 TB/s) 决定, Tensor Core 大量空闲等待数据。

在多 Agent 推理场景下, 访存受限是主要矛盾。Decode 阶段每步仅生成一个词元, 图形处理器 (Graphics Processing Unit, GPU) 仍须将模型权重自显存搬运至计算单元, 却只进行一次简单的矩阵-向量乘法, 计算强度很低, 性能几乎完全由权重自 HBM 读入的带宽决定。

多路并发带来的大批量与长上下文叠加, 又使 KV Cache 与模型权重同时驻留 HBM; 在长文本推理中, 读取 KV Cache 所耗带宽甚至可能超过读取模型权重, 从容量与带宽两侧同时加压。

此外, 近十年来 GPU 峰值算力 (常以万亿次浮点运算每秒计, Tera FLOPS, TFLOPS) 已提升数百倍, 而显存带宽仅提升约数十倍, 这一“剪刀差”使硬件性能上限越来越取决于数据搬运而非算术吞吐。

针对访存受限问题, 工程上有两条正交的优化路径。其一是算子融合, 面向逐元素及归约类访存受限算子: 逐算子执行时, 中间张量须反复写回 HBM 再读出, 产生本可在片上避免的无效往返; 将相邻访存受限算子融合为单内核, 可使中间结果留在静态随机存取存储器 (Static Random Access Memory, SRAM) 中, 直接削减 HBM 流量。其二是权重量化, 面向 Decode 阶段 GEMV 的带宽与容量双重约束; W4A16 (4 位整型 (INT4) 权重 + FP16 激活) 压缩权重占用以为大批量 KV Cache 腾出空间, INT4 权重加载流量降至 FP16 的 1/4, 从而提升 Decode 的有效带宽利用率, 但 INT4 反量化 GEMV 本身又成为新的热点, 需要高性能定制实现。

高效解决多 Agent 推理的性能问题, 核心在于针对访存受限特征定制高性能算子, 结合某种算子优化的算法, 对 HBM 访存模式、片上数据复用与硬件流水做精细的控制和编排。

在 Qwen3-32B 的推理链路中, QKV 投影、输出投影与前馈网络中的线性层

等规整矩阵乘（GEMM）已由 cuBLAS 提供接近峰值的优化实现，额外定制的收益极为有限。

未被 cuBLAS 覆盖、且在每一 Transformer 层中高频出现的热点共六类：带宽受限的 RMSNorm、RoPE、SwiGLU 与 GQA Decode，计算受限的 GQA Prefill，以及 Decode 阶段量化权重访存成为新热点的 W4A16 GEMV。这六类算子各具明确的性能瓶颈与可利用的结构化优化空间，是本文算子优化工作的直接针对目标。

1.1.3 高性能与开发速度的两难

算子级优化的价值已被工业界反复验证。FlashAttention^[5, 6, 7] 系列通过访存 IO 感知（IO-aware，按内存带宽约束组织计算）分块将注意力 HBM 流量从 $\mathcal{O}(N^2)$ 降至 $\mathcal{O}(N)$ ，成为业界的标准做法。然而，手写 CUDA（统一计算设备架构，Compute Unified Device Architecture）要实现算法逻辑，必须要结合具体的硬件信息和编程规范，需要经验丰富的工程师手写数千行模板元编程、手工异步拷贝原语（`cp.async`）流水与存储体交错（swizzle）布局管理。模型结构一旦变动，或者需要迁移到其他硬件（如新型 GPU），就需重写调度逻辑等大量代码，工程周期以月计，难以跟上模型、硬件迭代速度。

Triton^[8]（OpenAI 提出的 GPU 内核 Python DSL）降低了开发门槛，让程序员可以用类似 Python 的编程语言较简便地编写算子代码。但 Triton 将调度决策（线程绑定、内存布局、流水安排）与数据流逻辑混合在同一函数体，难以表达多级异步流水；面对需要精细控制 `cp.async` 时序与 swizzle 布局的复杂内核，其表达能力受限，导致无法细粒度地控制算子执行过程，性能受限。

这一高性能与开发速度的两难，使得针对新模型、新场景、新硬件快速迭代出高性能算子极为困难，是当前算子开发中的核心痛点之一。

1.1.4 TileLang：以解耦设计化解困境

本文选取国产领域特定语言 TileLang^[9] 作为算子实现工具，其关键设计在于：在编译器层将算法逻辑（`T.copy`、`T.gemm` 刻画“做什么”）与调度策略（`T.Pipelined`、`T.alloc_shared` 刻画“怎么执行”）显式解耦。布局推导、异步流水插入、Tensor Core 映射在编译阶段由编译器自动加入，开发者只需给出算法

描述，无需编写调度优化的细节；相同的算法描述可通过调整调度参数针对不同硬件自动生成优化实现，也可以自动搜索出最佳的参数。图 1-1 以 FlashAttention 内核为例，直观对比了手写 CUDA、Triton 与 TileLang 三种路线的代码抽象层次差异：自左至右，手写 CUDA 将算法与调度紧耦合；Triton 在块程序（block program）内混合表达数据流与调度；TileLang 则在编译器层将算法描述与编译变换遍（compiler pass, Pass）解耦、二者可独立演化。



图 1-1 三种 GPU 算子开发路线对比

本文以国产开源大模型 Qwen3-32B 为代表性实验模型、以 NVIDIA A100 为测试平台，依托 TileLang 构建面向多 Agent 场景的高性能推理原型，验证上述解耦设计在真实推理负载下的性能潜力与工程可行性。

1.2 国内外研究现状

1.2.1 高性能算子开发：从手写 CUDA 到领域 DSL

GPU 内核开发经历了从手写汇编/CUDA、厂家库（Vendor Library）到领域特定语言（Domain-Specific Language, DSL）的持续演进。NVIDIA cuBLAS（CUDA Basic Linear Algebra Subprograms, CUDA 基础线性代数子程序库）/cuDNN（CUDA Deep Neural Network library, CUDA 深度神经网络库）等厂家库在通用矩阵乘法

(General Matrix Multiply, GEMM)、卷积等规整负载上接近硬件峰值,但在很大程度上是闭源的,程序员无法修改、优化,很难实现深度的算子融合。CUTLASS^[10]提供了开源的模板化分块(tile)抽象,性能接近 cuBLAS,但仍要求开发者深入掌握 GPU 流水线细节,需要使用 C++ 模板元编程,非常复杂、代码量巨大,且与硬件代际强绑定。张量虚拟机(Tensor Virtual Machine, TVM)^[11, 12]、Halide^[13]、多级中间表示(Multi-Level IR, MLIR)^[14]等编译器框架以算法/调度分离为目标,在规整负载上实现了自动调优;但面对需要精细控制 `cp.async` 时序与 `swizzle` 布局的 FlashAttention 类内核,自动搜索空间过大,性能仍不及手工优化。这些工作的发展过程,展示了学界在推理算子开发中性能与可维护性之间持续寻求平衡的过程。

1.2.2 IO 感知优化与 FlashAttention

FlashAttention 通过分块在线 Softmax 将注意力中间矩阵的 HBM 足迹从 $\mathcal{O}(N^2)$ 降至 $\mathcal{O}(N)$,是 IO 感知算法设计的里程碑。FlashAttention-2 优化了线程束(warp, 32 线程成组调度)工作分区,FlashAttention-3 针对 NVIDIA Hopper 架构的 Warp Group 矩阵乘累加(Warp Group Matrix Multiply Accumulate, WGMMMA)/张量内存加速器(Tensor Memory Accelerator, TMA)指令实现了计算与访存的深度重叠。这一系列工作确立了“避免中间张量物化、让矩阵乘与 Softmax 在片上流水”的核心原则——对短/中序列 Decode 阶段可减少 HBM 往返,对长序列 Prefill 阶段则能让原本受显存容量限制的注意力在固定 SRAM 下完成。本文的 GQA Prefill/Decode 内核直接沿用这一设计原则,并在 TileLang 框架下以更简洁的算法描述实现等价逻辑。

1.2.3 切片编程语言的崛起与 TileLang 的定位

Triton 以块程序(block program)为基础,使 Python 风格的 GPU 内核描述成为可能,在 RMSNorm、GEMM 等算子上接近手写 CUDA 性能,显著降低了高性能内核的开发门槛。但如前所述,Triton 将调度决策与数据流逻辑混合在同一函数体,限制了复杂多级流水场景下的表达能力。ThunderKittens^[15]等工作以 C++ 模板封装 Hopper 上线程束级(warp-level)原语,性能接近峰值但可移植性较低。Welder^[16]、PipeThreader^[17]等则分别聚焦跨算子分块(tiling)与流水线自

动化，各有侧重。

TileLang 在上述工作的基础上，将数据流与调度在编译器层显式解耦，这是其与 Triton 最本质的差异：开发者以 `T.copy`、`T.gemm` 等原语描述算法逻辑，布局推导、异步流水插入、Tensor Core 映射作为独立 Pass 自动施加，使得同一算法描述可系统地搜索不同硬件上的最优调度配置。本文选择 TileLang，正是基于这一解耦设计在 Transformer 推理算子上展现出的可维护性与搜索效率优势，以及其作为国产 DSL 在自主可控推理栈中的战略价值。

1.3 本文主要工作

本文工作可概括为三个方面。在 Qwen3-32B 推理链路上以 TileLang 重写 RMSNorm、RoPE、GQA Prefill、GQA Decode、SwiGLU 与 W4A16 GEMV 六类热点算子，针对各自访存或计算瓶颈设计专项优化策略，并在 A100 上与 Triton 及 PyTorch（Python 深度学习框架）原生实现进行对比评估。设计三层推理原型系统，通过运行时算子替换将 TileLang 内核无侵入地嵌入推理链路，并以多 Agent 协作为场景实现交互展现层。最后，以微基准、端到端吞吐与消融实验对算子性能分层度量，并结合硬件利用率指标对各类算子的瓶颈与优化机制作归因分析。

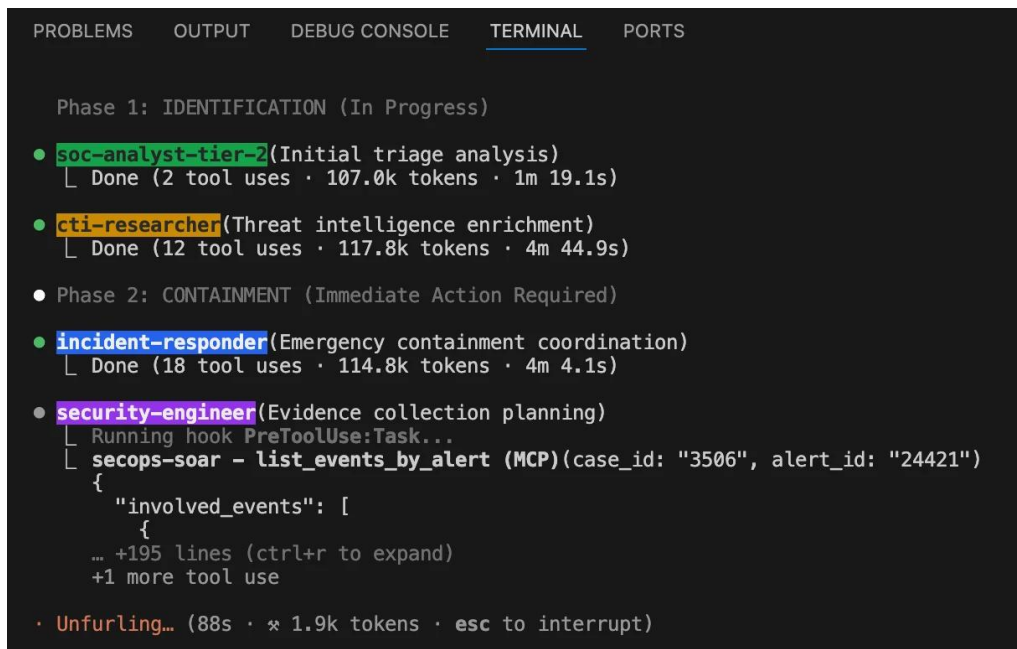
1.4 论文结构安排

本文共分五章。第一章为绪论，分析多 Agent 场景的负载特征与推理系统面临的三重挑战，阐述选取 TileLang 路线的动机与本文主要工作。第二章介绍支撑后文设计与实验的技术基础：多 Agent 推理场景、GPU 体系结构与 Roofline 模型、Qwen3-32B 热点算子分析、W4A16 权重量化技术与 TileLang 编程模型。第三章给出推理系统三层架构设计与六类算子的 TileLang 内核实现细节。第四章汇报算子微基准、端到端评估与消融实验结果，并给出利用率分析。第五章总结工作，展望后续方向。

2 相关技术与基础

2.1 多 Agent 推理场景

多 Agent 系统是当前大语言模型最重要的应用形态之一。以 Claude Code、OpenClaw 为代表的编程智能体（Agent）系统中，多个专用子 Agent 并发调用同一后端模型：代码审查、缺陷检测、测试生成、文档编写等任务被分配给不同 Agent，各自携带数千至数万词元的代码仓库片段作为上下文，并发地向推理后端发出请求（图 2-1）。图中示例 Agent 名称与十万词元级上下文取自 Claude Code 实际运行截图。



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Phase 1: IDENTIFICATION (In Progress)

• soc-analyst-tier-2 (Initial triage analysis)
  | Done (2 tool uses · 107.0k tokens · 1m 19.1s)

• cti-researcher (Threat intelligence enrichment)
  | Done (12 tool uses · 117.8k tokens · 4m 44.9s)

• Phase 2: CONTAINMENT (Immediate Action Required)

• incident-responder (Emergency containment coordination)
  | Done (18 tool uses · 114.8k tokens · 4m 4.1s)

• security-engineer (Evidence collection planning)
  | Running hook PreToolUse:Task...
  | secops-soar - list_events_by_alert (MCP)(case_id: "3506", alert_id: "24421")
  | {
  |   "involved_events": [
  |     {
  |       ... +195 lines (ctrl+r to expand)
  |     }
  |   ]
  | }
  | +1 more tool use

• Unfurling... (88s · ✖ 1.9k tokens · esc to interrupt)
    
```

图 2-1 多 Agent 系统运行示意

这一使用模式带来三重叠加压力，与传统单轮对话场景有本质区别。

第一，批量压力。多 Agent 并发使推理侧有效批量远大于传统单会话，解码（Decode）阶段的权重加载被多路请求分摊，HBM 带宽因而成为主要瓶颈。

第二，上下文压力。代码仓库与文档片段使提示长度常达 1024~2048 词元，KV Cache 在 Decode 阶段又持续增长至 4096~8192 词元，显存占用随并发数线性扩大。

第三，容量竞争压力。大参数模型的权重本身已占用大量显存，多卡分布式

部署下每张卡可用于 KV Cache 的空间极为有限，须借助权重量化（W4A16，4 位权重量化 + 16 位激活，Weight 4-bit Activation FP16）压缩权重以释放显存。

上述特征共同构成本文的研究动机：大批量下 Decode 阶段的算子性能（W4A16 GEMV、GQA Decode）与预填充（Prefill）阶段的计算效率（GQA Prefill、RMSNorm、RoPE）是决定智能体（Agent）推理系统端到端吞吐的关键路径，也是本文算子优化工作的直接针对目标。

2.2 GPU 体系结构与并行计算模型

2.2.1 存储层级与带宽



图 2-2 GPU 存储层级（A100）

高性能算子优化的物理基础是 GPU 的多级存储层级，如图 2-2 所示。以 NVIDIA A100 SXM4-80GB 为例，自外向内首先是第二代高带宽显存增强版（High Bandwidth Memory 2E，HBM2e）全局显存，容量 80 GB，带宽约 2.0 TB/s，访问延迟约 400~600 个时钟周期，模型权重与 KV Cache 驻留于此，是推理性能的主要瓶颈。

其内为二级缓存（Level-2 Cache，L2），由所有流式多处理器（Streaming Multiprocessor，SM）共享，容量 40 MB，带宽约 4~5 TB/s，可部分摊薄 HBM 访问延迟。

再向内为每 SM 私有的共享内存（Shared Memory）与一级缓存，在 A100 上可配置为 192 KB，带宽约 20 TB/s，延迟约 20 个周期，是片上数据重用的核心载体。最内层为寄存器文件，每 SM 256 KB，每线程最多 255 个 32-bit 寄存器，带宽最高而容量最小，一旦发生寄存器溢出（register spill）将带来显著性能回退。

HBM 与 SRAM 的带宽比约为 1:10，而容量比约为 400:1。这一不对称性是 IO 感知优化的出发点：将热数据保留在 SRAM 片上完成计算，避免反复落回 HBM，是 FlashAttention 类算法的核心策略，也贯穿本文所有算子的设计。

2.2.2 线程层级与 Tensor Core

CUDA 采用网格(Grid)→线程块(Block)→线程束(Warp)→线程(Thread)四级线程层级，Warp(固定 32 线程成组调度)是 GPU 调度的最小单位。Block 内线程通过共享内存通信，Block 间不能直接同步。Block 大小影响占用率(occupancy)：过低则 SM 无法用 warp 切换来隐藏访存延迟，过高则共享内存与寄存器压力增大。

Ampere 架构的 Tensor Core 支持 BF16 (Brain Float 16) /FP16 输入、FP32 累加的矩阵乘累加 (Matrix Multiply-Accumulate, MMA)，单个 Warp 每周期可完成 $16 \times 8 \times 16$ 矩阵乘，A100 的 BF16 峰值因此达到 312 TFLOPS。Hopper 引入了更大粒度的 WGMMMA (Warp Group MMA, 线程束组 MMA) 指令。触发 Tensor Core 要求数据满足严格的布局约束——主维 (leading dimension, 矩阵线性存储时的行/列跨度) 须为 16 的倍数，共享内存中的分块 (tile) 需经 swizzle 重排以避免存储体冲突 (bank conflict, 多线程同时访问共享内存同一存储体引发的串行化)。这些约束在手写 CUDA 中需开发者显式维护，是内核开发中主要的工程负担之一；TileLang 的布局推导 Pass 将其自动化。

2.2.3 Roofline 模型

Roofline 模型^[18]以算术强度 (Arithmetic Intensity, AI = 浮点运算次数 FLOPs / HBM 访存字节数) 为横轴，将算子性能上限表达为：

$$\text{Performance} \leq \min(\text{Peak FLOPS}, \text{Bandwidth} \times \text{AI}) \quad (2-1)$$

该模型将性能瓶颈分为两类：AI 较低时性能上限由 HBM 带宽决定（带宽受限，memory-bound），优化目标是减少 HBM 流量；AI 较高时由峰值算力决定（计算受限，compute-bound），优化目标是提高 Tensor Core 利用率。两段的分界点即屋脊点（ridge point），A100 BF16 下约为 $312 \text{ TFLOPS} / 2.0 \text{ TB/s} \approx 156 \text{ FLOP/Byte}$ 。算术强度低于屋脊点的算子通常受带宽限制，高于屋脊点的算子通常受计算能力限制。

2.3 Qwen3 推理中的热点算子

2.3.1 模型结构与算子组成

Qwen3-32B^[19] 是稠密 Transformer（变换器）架构，共 64 层，每层由注意力模块与前馈网络（Feed-Forward Network，FFN）构成。关键超参见表 2-1；表中取值作为后文微基准与系统实验的统一参照口径。

表 2-1 Qwen3-32B 关键结构超参

参数	取值
隐藏维度 d_{model}	5120
注意力头数 H_q	64
KV 头数 H_{kv}	8（GQA，group size $G=8$ ）
头维度 D	128
FFN 中间维	27648（SwiGLU）
层数 L	64
最大上下文	128K tokens

每层前向计算由以下算子序列构成：Input RMSNorm → 查询-键-值（Query-Key-Value，QKV）投影（通用矩阵乘，GEMM）→ RoPE → GQA Attention → 输出投影（GEMM）→ Post-Attn RMSNorm → SwiGLU FFN（ $2 \times \text{GEMM} + \text{逐元素运算} + \text{GEMM}$ ）。QKV 投影、输出投影与 FFN 中的 FP16 GEMM 由 cuBLAS 处理；在 W4A16 推理路径下，Decode 阶段多层感知机（Multi-Layer Perceptron，MLP）的 gate/up/down 投影可替换为 INT4 权重 + FP16 激活的矩阵-向量乘（GEMV），由本文实现的 W4A16 GEMV 内核承担。综合起来，本文在算子层聚焦六类对象：RMSNorm、RoPE、GQA Prefill、GQA Decode、SwiGLU 与 W4A16 GEMV。

为何是这六类而非其他？推理链路中的 QKV 投影、输出投影与 FFN 线性层均是规整 GEMM，由 cuBLAS 在 A100 上提供接近峰值的实现，本文不重复造轮子。余下的非 GEMM 热点，恰好落在上述六类上：RMSNorm、RoPE、SwiGLU、GQA Decode 四类均属带宽受限归约或逐元素算子，算术强度远低于屋脊点，每步推理各调用数十次，HBM 访问累积可观；GQA Prefill 在长序列下进入计算受限区，同时有中间矩阵物化的显存压力，需要分块 Softmax；W4A16 GEMV 则是权重量化后 Decode 阶段新出现的热点，朴素实现中 INT4 解包与显存访问成为瓶颈。

这六类算子在 Qwen3-32B 每一 Transformer 层中高频反复出现，兼具明确的瓶颈成因与可利用的结构性优化空间，是制约多 Agent 场景端到端吞吐的关键非 GEMM 热点，也构成第三章算子设计与实现的对象全集。

2.3.2 GQA 注意力：Prefill 与 Decode 的负载差异

Qwen3 采用分组查询注意力（GQA）^[20]， $G=H_q/H_{kv}=8$ ，即每 8 个 Q 头共享 1 组 KV 头，KV Cache 体量相比多头注意力（Multi-Head Attention, MHA）压缩 8 倍。注意力计算为：

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{D}}\right)V \quad (2-2)$$

在 Prefill 阶段（ $S_q=S_{kv}=N$ ）， $QK^\top \in \mathbb{R}^{N \times N}$ ，计算量 $\mathcal{O}(BH_qN^2D)$ ， $S=4096$ 时算术强度约 1820 FLOP/Byte（推导见 2.3.4 节），已处于计算受限区；若直接物化 $N \times N$ 中间矩阵将消耗 $\mathcal{O}(N^2)$ HBM，FlashAttention 类分块 Softmax 成为必要手段（主要为节省显存，而非为降低带宽压力）。

在 Decode 阶段（ $S_q=1$ ），每步仅新增 1 个词元的查询，却须读取全量 KV Cache（ $4BS_{kv}H_{kv}D$ 字节，K 与 V 各一次 FP16），FLOP 约 $2BH_qS_{kv}D$ ，算术强度约 $H_q/(2H_{kv}) = 64/16 = 4$ FLOP/Byte，属典型带宽受限场景，内核设计须侧重沿 S_{kv} 维度的 Split-K（沿 K 维切分并行累加）与合并访存（coalesced）的 KV 读取。

2.3.3 RMSNorm、RoPE 与 SwiGLU

RMSNorm^[21] 对每行 $x \in \mathbb{R}^{d_{\text{model}}}$ 做均方根（Root Mean Square, RMS）归一化：

$$\text{RMSNorm}(x)_i = \frac{x_i}{\sqrt{\frac{1}{N} \sum_j x_j^2 + \epsilon}} \cdot \gamma_i \quad (2-3)$$

每层调用两次（输入归一化 + 后注意力归一化）。朴素两个内核实现（求 RMS + 乘权重）需读 x 两次（ $4MN$ B）+ 写一次（ $2MN$ B）；融合单内核两阶段遍历（第一阶段求 inv_std ，第二阶段乘以 γ ）读 x 仍需两次，但 inv_std 在寄存器片（fragment）上保留，无需落回 HBM，总 HBM 流量降至 $3MN$ B，是典型的归约算子 IO 优化场景。

RoPE (rotate_half)^[22] 对 Q, K 施加旋转位置编码，Qwen3 采用 rotate_half 约定：

$$y_j = x_j \cos \theta_j - x_{j+D/2} \sin \theta_j, \quad y_{j+D/2} = x_j \sin \theta_j + x_{j+D/2} \cos \theta_j \quad (2-4)$$

每层对 Q, K 各调用一次，是典型的逐元素（elementwise）算子。

SwiGLU^[23] 是 FFN 的激活函数， $\text{SwiGLU}(x_1, x_2) = \text{silu}(x_1) \odot x_2$ ，其中 $\text{silu}(x) = x \cdot \sigma(x)$ 。朴素实现将 silu 与逐元素乘拆成两次内核调用（3 次读 + 2 次写），融合实现合为一次（2 次读 + 1 次写），理论可节省约 40% HBM 流量。

自回归 Decode 每步仅扩展一个词元，MLP 中 gate、up、down 三个线性层在形状上退化为矩阵-向量乘（GEMV），算术强度极低，性能主要由权重自 HBM 读入的带宽决定。

W4A16 将权重以 INT4 存储、激活保持 FP16，在几乎不改变上层语义的前提下把权重访存压缩为 FP16 的约四分之一，是 Agent 场景下在有限显存内容纳大批量与长 KV 的常见工程选项。其热点内核为带逐组反量化与 LOP3 快速解包的 GEMV，与前五类 FP16 算子并列，构成本文第六类优化对象。

2.3.4 算术强度分析

结合 Roofline 模型，对五类算子的算术强度（AI）逐一推导（FP16，融合实现， $M=B \cdot S$ ）。

对形状 $[M, N]$ 的 RMSNorm，融合两阶段内核两次读 x ($4MN$ B) 加一次写 o ($2MN$ B)，总计 $6MN$ B，计算量约 $3MN$ FLOP（平方、求和、rsqrt、乘法），故 $AI = 3/6 = 0.5$ FLOP/Byte。

RoPE 的 rotate_half 每对元素 $(x_j, x_{j+D/2})$ 执行 6 FLOP，HBM 访问含读 x ($2MD$ B)、读 $\cos/\sin$ 表 ($2MD$ B)、写 y ($2MD$ B)，总计 $6MD$ B，故 $AI = 0.5$ FLOP/Byte。

SwiGLU 融合实现每元素约 5 FLOP（silu 约 4 FLOP 加乘法 1 FLOP），HBM 访问为读 gate、读 up 与写出，合计 $6MN$ B，故 $AI \approx 0.83$ FLOP/Byte。

GQA Decode 每步 FLOP 约 $2BH_qS_{kv}D$ ，KV 读取量为 $4BS_{kv}H_{kv}D$ B， $AI \approx H_q/(2H_{kv}) = 4$ FLOP/Byte（不计 Q 读取）。

GQA Prefill 在 FlashAttention 风格下仅读写 Q, K, V, O 各一次 $((2H_q+2H_{kv})SD$ FP16 字节)， $FLOPs \approx 4H_qS^2D$ （causal 减半）， $AI = \frac{H_qS^2D}{(H_q+H_{kv})SD} = \frac{H_qS}{H_q+H_{kv}} = \frac{GS}{G+1} \approx \frac{8 \times 4096}{9} \approx 3641$ FLOP/Byte（non-causal, causal 约减半 ≈ 1820 ），远超屋脊点，属于计算受限。

五类算子的 AI 汇总于表 2-2。

表 2-2 五类算子算术强度汇总

算子	AI (FLOP/Byte)
RMSNorm	≈ 0.5
RoPE	≈ 0.5
SwiGLU（融合）	≈ 0.83
GQA Decode	≈ 4
GQA Prefill ($S=4096$)	≈ 1820 (causal)

图 2-3 将五类算子标注在 Roofline 图中。图中折线实线为 A100 BF16 的理论性能上界：左段斜线受 HBM 带宽（2 TB/s）限制，性能随算术强度线性增长；右段水平线受峰值算力（312 TFLOPS）限制，性能不再增长；竖向虚线为屋脊点（ ≈ 156 FLOP/Byte），是两段的分界。各算子标记点落在曲线上，横坐标为其理论算术强度。GQA Prefill ($AI \approx 1820$ FLOP/Byte) 超出图横轴范围，标注于右侧边界附近；其余四类均在带宽受限区。

可以看到，RMSNorm、RoPE、SwiGLU 三类算子的 AI 均不足 1，聚集在图左侧最靠近原点的位置，远离屋脊点；GQA Decode 的 $AI \approx 4$ ，仍处于带宽受限

区。GQA Prefill 是唯一的计算受限算子，其 $AI \approx 1820$ (causal, $S=4096$)，远超屋脊点——此时的瓶颈是 Tensor Core 利用率而非 HBM 带宽，实验实测有效算力达 153 TFLOPS（约 49% 峰值算力）与此一致。

对 RMSNorm、RoPE、SwiGLU、GQA Decode 四类算子，优化瓶颈是 HBM 读写次数，IO 感知优化策略（算子融合、片上数据重用）是核心手段；对 GQA Prefill，Tensor Core 利用率与内存容量（分块 Softmax 压缩 $\mathcal{O}(S^2)$ 临时存储）才是主要设计约束。

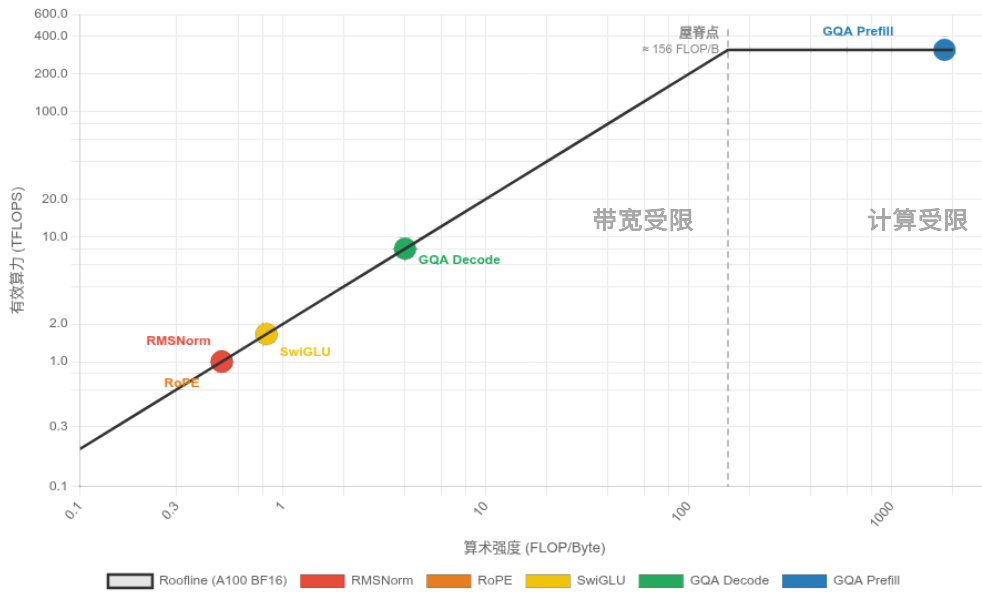


图 2-3 五类算子的 Roofline 分布

2.4 W4A16 权重量化技术

随着大语言模型参数规模的扩大，显存容量与带宽已成为推理部署的核心瓶颈。权重量化通过降低参数的存储位宽，在不引入额外硬件的前提下同时缩减显存占用与 HBM 访问量，是当前工业推理栈（vLLM、TensorRT-LLM 等）广泛采用的加速手段。

2.4.1 量化方案选型：W4A16 与 W8A8

主流量化方案可分为两类。W8A8^[24]（权重与激活均量化至 INT8）依赖 INT8 Tensor Core，在 Prefill 等计算受限场景下可提升算力利用率，但需对激活动态量化（运行时逐词元计算尺度因子），引入额外归约开销，且激活值的长尾分布易

导致精度退化。

W4A16^[25, 26] (INT4 权重 + FP16 激活) 则将权重存储位宽压缩至 4 bit，权重 HBM 读取量降至 FP16 的 $\frac{1}{4}$ ，激活保持 FP16 无需动态量化。对于 Decode 阶段的 MLP 投影层，算术强度约为 1 FLOP/Byte，远低于 A100 屋脊点，属于强访存受限场景——此时延迟近似正比于权重字节数，W4A16 的权重压缩直接等比缩短访存时间。如图 2-4 所示，相比 W8A8，W4A16 在访存受限的生成场景下具备更低的带宽压力与更简洁的运行路径，是本文的首选量化方案。

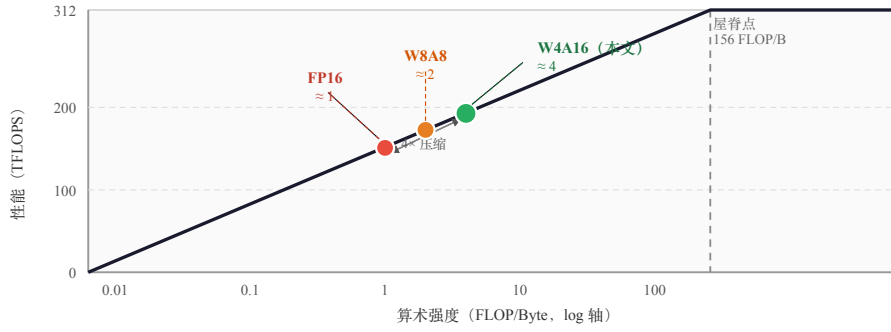


图 2-4 Decode 阶段 W4A16 与 W8A8 对比

2.4.2 per-group 非对称量化

为控制 INT4 量化的精度损失，本文采用 per-group 非对称量化 (group_size=128)：将每行权重向量按 128 个元素为一组，独立计算尺度因子 s 与零点 z ：

$$\hat{W}_{ij} = \text{clamp}\left(\text{round}\left(\frac{W_{ij}}{s} + z\right), 0, 15\right) \quad (2-5)$$

反量化时 $W_{ij} = (\hat{W}_{ij} - z) \cdot s$ 。per-group 量化相比 per-channel 可将量化误差控制在更细粒度，在 7B 以上模型上精度损失通常可忽略。

2.4.3 LOP3 快速反量化

GPU 不支持 4 bit 原生数据类型，INT4 权重须在运算前解包至 FP16。朴素解包路径（移位 + 掩码 + 整数转换 + 浮点转换）需 5–6 条指令。本文采用与 MARLIN^[27] 类似的 LOP3 指令 (lop3.b32) 技巧——单条指令可对三个 32 bit 寄存器执行任意布尔函数——利用 FP16 的 IEEE 754 位结构，将 nibble 直接映射至 FP16 尾数位，以约 4 条 PTX 指令完成解包，比朴素路径节省约 60% 解码

指令数，是 W4A16 GEMV 内核高效实现的关键。

2.5 TileLang 编程模型

2.5.1 可组合切片抽象与解耦设计

TileLang 的核心设计是数据流与调度的显式解耦，用户通过两类正交原语描述内核。数据流一侧用 `T.alloc_shared`、`T.alloc_fragment` 声明三级存储中的缓冲，用 `T.copy(src, dst)` 声明 tile 级搬运，用 `T.gemm(A, B, C)` 声明 tile 级矩阵乘，并以 `T.reduce_sum`、`T.cast`、`T.Parallel` 等刻画规约、类型转换与并行循环。调度一侧则用 `T.Kernel(grid, threads)` 声明 Grid 与 Block，用 `T.Pipelined(range, num_stages=n)` 声明软件流水线，并以 `@tilelang.autotune`、`@tilelang.jit` 分别声明参数搜索空间与 JIT 编译入口及 Pass 配置。

这两类原语相互独立——同一份数据流描述可以绑定不同的调度参数，由编译器在不同参数组合下生成不同的优化实现，而无需修改算法逻辑。与 Triton 的块程序将调度决策（BLOCK_M、线程绑定、内存布局）直接写入函数体不同，TileLang 的解耦设计使算法逻辑对调度变化透明。

2.5.2 编译管线与核心 Pass

TileLang 在 TVM IR 层按顺序施加一串核心编译 Pass。InstructionAnnotation 识别 `T.copy`、`T.gemm` 等 tile op 并标注目标指令类型（`cp.async`、WMMA、TMA 等）。PipelinePlanning 分析 `T.Pipelined` 块内数据依赖，以确定多缓冲阶段数与屏障插入点。

LayoutInference（布局推导）逐步推导各 tile buffer 的每线程访问模式，自动施加 swizzle（如 $(i, j) \rightarrow (i, j \oplus (i \% 8))$ ）消除共享内存存储体冲突（bank conflict），并保证 Tensor Core 所需对齐。LowerTileOp（张量化映射）将 `T.gemm` 与目标 MMA 形状匹配，在 A100 上发射 `mma.sync.aligned.m16n8k16.f32.f16.f16.f32`（Ampere WMMA），在 H100 上发射 WGMMMA。

InjectSoftwarePipeline 将 `T.Pipelined` 展开为多缓冲循环并插入 `cp.async` 与 `cp.async.wait_group`，使计算与 HBM→SMEM 搬运重叠。ThreadSync 在依赖处插入 `__syncthreads` 或 warp-level¹⁶ 屏障。最后经标准 TVM Pass 完成存储重写、内建原语下降（intrinsic lowering）与并行线程执行（Parallel Thread Execution, PTX）代码生成。

布局推导与张量化两个阶段是 TileLang 相比 Triton 最显著的自动化优势：Triton 中存储体冲突消除与 Tensor Core 布局对齐需要开发者手动设置 `.contiguous()`、`trans()` 等调用，而 TileLang 将其纳入编译器 Pass，无需用户介入。图 2-5 展示了完整编译管线及各 Pass 的职责划分；图中蓝色方框标示编译器自动施加的核心优化 Pass。

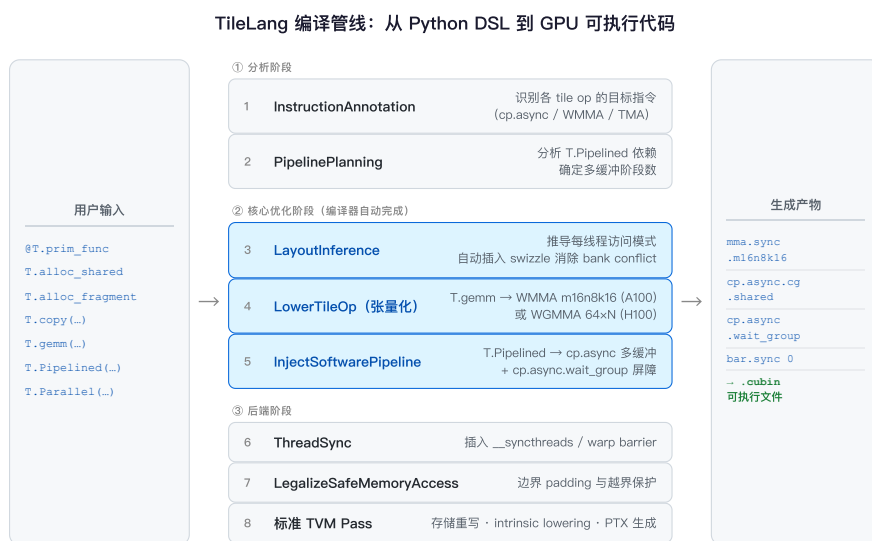


图 2-5 TileLang 编译管线

2.5.3 与 Triton 的核心差异

表 2-3 TileLang 与 Triton 对比

维度	Triton	TileLang
存储模型	隐式两级（全局/寄存器）	显式三级（HBM/SMEM/Fragment）
内存布局	开发者手动控制	LayoutInference Pass 自动推导
多级流水	需手写状态机	T.Pipelined 声明后 Pass 自动展开
张量化	tl.dot 自动推断	T.gemm + LowerTileOp 显式映射
调度与数据流	混合在同一函数体	编译器层显式解耦

TileLang 与 Triton 的本质差异不在于语法，而在于抽象层次与编译器责任边界。如表 2-3 所示，二者在存储模型、内存布局与调度表达三个维度存在根本区别。

这一解耦的工程价值在于：当算法结构变化（如 GQA 的头映射参数从 $G=1$ 变为 $G=8$ ）时，只需修改数据流部分，调度优化（swizzle、流水、张量化）由编译器重新推导，无需逐一手工调整。

2.6 本章小结

本章从使用场景、硬件、模型与优化工具四个角度建立了技术背景：多 Agent 并发推理场景确定了大批量、长上下文的负载约束；GPU Roofline 模型给出访存/计算受限的判别框架；Qwen3-32B 算子分析与 W4A16 量化技术明确了优化对象与显存压缩策略；TileLang 编程模型为后续算子实现提供工具基础。

3 系统设计实现

3.1 系统总体架构

本章基于 TileLang 编程模型，设计并实现一套面向多 Agent 推理场景的 Qwen3-32B 推理系统。系统采用“算子内核 / 推理调度 / 交互展现”三层解耦架构：算子内核层封装六类基于 TileLang 的高性能内核，覆盖带宽受限与计算受限两类算子场景；推理调度层通过运行时替换机制将内核无侵入地注入推理流程，使算子层的加速得以传导至系统可测量的吞吐与延迟指标；交互展现层在统一接口之上支持多 Agent 并发推理与后台性能监控，为系统在真实多租户场景下的验证提供完整闭环。三层之间职责清晰、依赖单向，便于逐层验证与独立替换。

系统的主要性能增益来自算子内核层。第二章的 Roofline 分析揭示了各算子的具体瓶颈：RMSNorm、RoPE、SwiGLU 与 GQA Decode 受限于 HBM 访存带宽，GQA Prefill 受限于 Tensor Core 利用率不足，W4A16 GEMV 则以 INT4 权重压缩拓展 Decode 阶段的吞吐上限。本章基于 TileLang 编程模型针对上述瓶颈逐一设计内核，并最终在完整推理系统、真实模型权重与推理负载下得到端到端验证。

3.1.1 三层架构总览

系统采用“算子内核 / 推理调度 / 交互展现”三层解耦设计，自下而上分别负责高性能计算、推理流程管理与用户交互，每层职责独立、接口明确，如图 3-1 所示。

算子内核层是系统最底层，由六类 TileLang 算子构成，每类对外暴露一个与 PyTorch 语义完全等价的包装函数，将输入 shape 映射到最优 tile 配置并调用 JIT 编译缓存的内核对象；TileLang 编译管线（数据流与调度解耦、自动布局推导、软件定义流水线）在编译期自动施加，算子开发者只需描述算法逻辑。推理调度层基于 HuggingFace transformers 加载 Qwen3-32B 权重，通过运行时 monkey-patch 将 TileLang 内核注入计算图，并根据序列长度 S_q 在 Prefill 与 Decode 路径间动态切换，对上层完全透明；3.3 节将详细说明该机制。交互展现

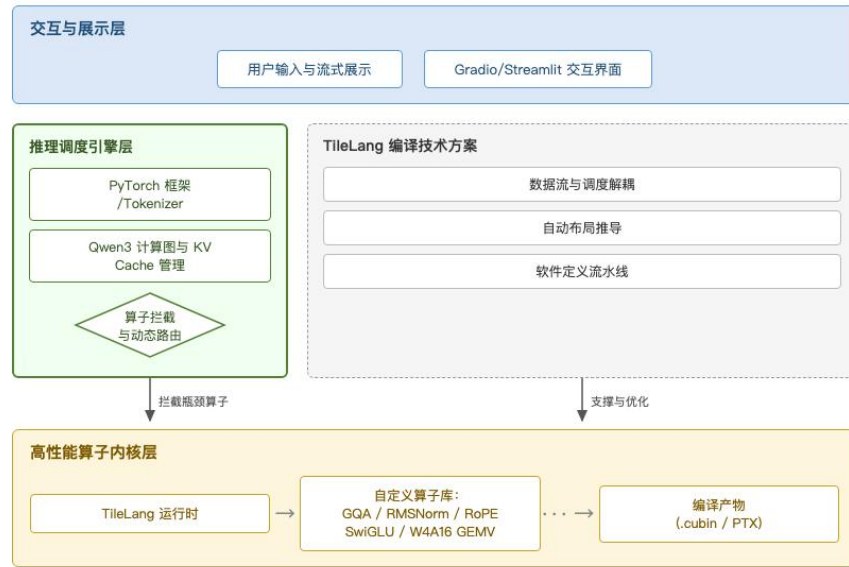


图 3-1 推理系统三层架构

层通过 `generate(prompt, backend, batch_size)` 接口与推理引擎解耦，支持对话交互、离线批处理、多 Agent 协作等多种上层应用；本文以多 Agent 协作为具体场景，实现了用户侧并发推理与服务器后台监控两个功能模块，3.4 节将展开介绍。

3.1.2 设计目标与系统边界

第二章的分析表明，Qwen3-32B 推理链路中的 RMSNorm、RoPE、SwiGLU、GQA Decode 四类算子处于带宽受限区域，优化核心是减少 HBM 访问；GQA Prefill 则处于计算受限区域，优化核心是 Tensor Core 利用率与分块 Softmax 节省中间显存；W4A16 GEMV 是 Decode 阶段 MLP 投影层的带宽受限热点，优化核心是 INT4 权重的高效反量化与计算访存流水重叠。本章的目标是设计一套推理原型系统，使 TileLang 实现的高性能算子能够无缝替换进 Qwen3-32B 的推理链路，并支持与 Triton、PyTorch 原生实现的公平对比。

系统在硬件上固定为 NVIDIA A100 SXM4-80GB (SM_80)；TileLang IR 虽具后端无关性，本文实测仅覆盖 A100，不涉及 Hopper 或国产加速器。精度上采用 FP16 权重与激活、FP32 中间累加，与 Qwen3-32B 官方权重格式一致，并额外支持 W4A16 GEMV (INT4 权重 + FP16 激活) 以评估权重量化对吞吐的潜力，该路径仅用于 MLP 的 Decode 投影层。

评测环境共 8 张 A100，推理阶段采用 2 卡数据并行；多卡张量并行与服务化调度不在讨论范围内，以免系统层噪声干扰算子级对比。端到端评测采用静态批处理，批量设置为 $B \in \{4, 8\}$ 。对比基线取 Triton 与 PyTorch 即时执行，不与 vLLM、TensorRT-LLM 等工业栈并列，因后者含连续批处理、多卡调度等额外因素，难以公平归因到算子层。

3.2 算子优化实现

本节详细介绍六类热点算子的 TileLang 实现方案。这六类算子覆盖了 Qwen3-32B 推理链路中的主要计算热点，在第二章 Roofline 分析的指导下，针对各自的性能瓶颈采取了差异化的优化策略。

算子分类与优化方向如下。六类算子可分为三组：（1）带宽受限归约/逐元素类——RMSNorm、RoPE、SwiGLU，受限于 HBM 带宽，优化重心在削减 HBM 访问次数，采用算子融合与片上中间结果复用；（2）注意力类——GQA Prefill 在长序列（ $S=4096$ ）下进入计算受限区，采用分块 Softmax 与 TileLang T.Pipelined 流水线；GQA Decode 为单 token 场景，再次回到带宽受限，采用 Split-K 分块缓解数据加载瓶颈；（3）量化矩阵乘类——W4A16 GEMV 承担 Decode 阶段的量化权重矩阵乘，利用 LOP3 指令将反量化与计算流水重叠，降低 INT4 解码开销。

所有算子拥有一套公共优化基础。所有算子均通过 `@tilelang.autotune` 装饰器搜索最优 tile 尺寸（`blk_m`、`blk_k` 等参数），并依赖 TileLang 的 `LayoutInference` Pass 自动推断 shared memory 布局以消除 bank conflict，无需手动 padding。每个算子内核均经过数值正确性验证（对齐 PyTorch 参考实现，误差在 FP16 精度范围内），再进入性能调优阶段。

3.2.1 RMSNorm 算子

3.2.1.1 算法与访存模式

RMSNorm 公式见式 (3-1)：

$$\text{RMSNorm}(x)_i = \frac{x_i}{\sqrt{\frac{1}{N} \sum_{j=1}^N x_j^2 + \epsilon}} \cdot \gamma_i \quad (3-1)$$

其中 $N = 5120$ （Qwen3-32B 隐藏维度）， $\epsilon = 10^{-6}$ ， γ 为可训练权重。

RMSNorm 需要两次遍历输入 x : 第一次求 $\sum x_j^2$ 并计算 inv_std , 第二次将 x_i 乘以 inv_std_i 与 γ_i 。本文的融合两阶段内核两次从 HBM 读入 x (共 $2 \times 2MN = 4MN$ B, 每次 FP16), 写出 O ($2MN$ B), 加上读权重 γ ($2N$ B, 可忽略), HBM 总流量 $\approx 6MN$ B。关键优化在于: inv_std (长度 M 的向量) 在阶段一结束后保留在寄存器 fragment 上, 阶段二直接复用, 不涉及 HBM 往返。相比朴素两个内核拆分实现, 融合路径避免了 (a) 两次内核启动与调度开销, 以及 (b) M 个 inv_std 值落回 HBM 再读回的往返。

3.2.1.2 TileLang 两阶段实现

本文的内核 `rmsnorm_mul_weight_fwd` 采用两阶段结构。Kernel Grid 为 $\lceil M/\text{blk_m} \rceil$, 每 block 处理 blk_m 行。

阶段一沿 N 维以 blk_k 为步长循环, 每次将 $[\text{blk_m}, \text{blk_k}]$ 行块自 HBM 拷入共享内存, 再以 FP32 将 x^2 累加至寄存器 fragment `sq`:

算法 3-1 · RMSNorm 阶段一

输入: 张量分块 X , 块维度 blk_m 与 blk_k

输出: 每行的标准差倒数 inv_std

```

1  alloc xs[blk_m, blk_k] in shared memory
2  alloc sq[blk_m, blk_k] in register, init to 0
3  for kk = 0 to num_k_step - 1 do
4      load X[row_tile, kk*blk_k : (kk+1)*blk_k] -> xs    // HBM -> SMEM
5      for each (i, j) in parallel do
6          sq[i, j] += cast(xs[i, j], fp32)^2
7  reduce_sum(sq, dim=1) -> row_sum_sq
8  for each i in parallel do
9      inv_std[i] = rsqrt(row_sum_sq[i] / N + eps)           // 保留在寄存器, 不落 HBM

```

阶段二再次沿 N 维遍历, 将各 $[\text{blk_m}, \text{blk_k}]$ 行块乘以 inv_std 与 γ 后写回输出:

算法 3-2 · RMSNorm 阶段二

输入: 张量分块 X , 权重 Weight , 标准差倒数 inv_std

输出：规范化输出 Out

```

1  alloc xs[blk_m, blk_k], ws[blk_k] in shared memory
2  for kk = 0 to num_k_step - 1 do
3      load X[row_tile, kk*blk_k : (kk+1)*blk_k] -> xs
4      load Weight[kk*blk_k : (kk+1)*blk_k]      -> ws
5      for each (i, j) in parallel do
6          y = cast(xs[i,j], fp32) * inv_std[i] * cast(ws[j], fp32)
7          Out[row_tile+i, kk*blk_k+j] = cast(y, fp16)

```

两次遍历共享 `inv_std` 且均在片上完成，不涉及 HBM 往返。

AutoTune 搜索维度为 $(\text{blk_m}, \text{blk_k}, \text{threads})$ ，共 256 个 config。对 Qwen3 $N=5120$ ，最优 config 倾向于较大的 K-tile ($\text{blk_k}=1024$) 以减少循环次数。

3.2.1.3 微架构实证分析

为验证上述两阶段融合设计在硬件上的收益，使用 NVIDIA Nsight Compute (GPU 内核级剖析工具，命令行常称 `ncu`) 在形状 $M=8192, N=5120$ (fp16) 下对 RMSNorm 做计数器对比，结果见表 3-1。

表 3-1 RMSNorm ncu 计数器对比

指标	TileLang	Triton
Kernel 延迟 (μs)	98.2	130.4
Grid $\times$ Block	8192×512	8192×128
DRAM 读取 (MB)	84.0	99.9
DRAM 写入 (MB)	84.0	87.5
DRAM 读占峰值比例 (%)	42.0%	37.6%
L2 命中率	62.9%	65.1%
SM 吞吐占峰值比例 (%)	34.7%	39.2%
SM 占用率 (Occupancy, %)	85.7%	94.2%
Warp 停顿原因 Top-3 (平均每周期停滞 warp 数)		
Long Scoreboard (等待 HBM)	20.9	26.4
Barrier (<code>__syncthreads</code>)	4.77	1.38
未选中 (Not Selected)	2.40	1.49

从表中数据可以看出：TileLang 的 DRAM 读取量仅 84.0 MB (接近理论下界 $5120 \times 2 \times \text{fp16} = 80 \text{ MB}$)，而 Triton 为 99.9 MB，多余的 16 MB 对应 `scale` 参数被反复从 HBM 加载的开销；同时 TileLang 的 Long Scoreboard 停顿指标 (20.9)

低于 Triton (26.4)，说明减少访存次数直接缩短了 warp 等待时间。

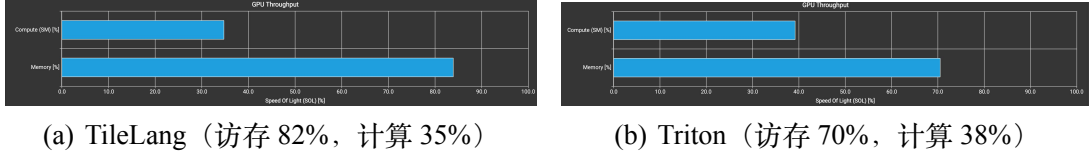


图 3-2 RMSNorm ncu 光速分析

图 3-2 的 ncu 光速 (Speed of Light) 视图定量说明了内核所用资源占峰值的比例：访存条带 (82% 对 70%) 反映 HBM 带宽利用率，计算条带 (35% 对 38%) 反映 SM 计算利用率。两者访存 > 计算，确认 RMSNorm 为带宽受限算子；TileLang 访存侧占峰值比例更高 (82% vs 70%)，源于两阶段融合内核借助 T.Pipelined 将 HBM 预取与计算持续重叠，内存控制器无需等待内核启动间隙，带宽利用更充分。

3.2.2 RoPE 算子

3.2.2.1 算法原理

RoPE (Rotary Position Embedding) 在每个注意力头内部对通道维做二维旋转，将位置信息注入 Q/K 而不引入额外参数。设头维为 D ，记 $\text{half} = D/2$ ，对每一对通道 $(x_j, x_{j+\text{half}})$ 施加与位置相关的旋转角 θ_j ，其计算为

$$\begin{bmatrix} y_j \\ y_{j+\text{half}} \end{bmatrix} = \begin{bmatrix} \cos \theta_j & -\sin \theta_j \\ \sin \theta_j & \cos \theta_j \end{bmatrix} \begin{bmatrix} x_j \\ x_{j+\text{half}} \end{bmatrix}, \quad j = 0, \dots, \text{half} - 1. \quad (3-2)$$

等价写成逐元素形式即

$$y_j = x_j \cos \theta_j - x_{j+\text{half}} \sin \theta_j, \quad y_{j+\text{half}} = x_j \sin \theta_j + x_{j+\text{half}} \cos \theta_j. \quad (3-3)$$

本文实现采用 Qwen3 的 rotate_half 配对方式，对输入张量逐行应用上述变换并写回输出。

3.2.2.2 TileLang 实现

内核 rope_rotate_half_fwd 的 Grid 为 $\lceil M/\text{blk_m} \rceil$ ，其中 $M = B \cdot S \cdot H$ 。每 block 处理 blk_m 行。

若将整行 $[\text{blk_m}, D=128]$ 置于单一共享内存缓冲，访问 $\text{xs}[i, j]$ （低半）与 $\text{xs}[i, j+64]$ （高半）时在 $D=128$ FP16 布局下会映射到同一 bank（64 个 FP16 恰为 128 字节，即 32 个 bank 各 4 字节），形成 2-way bank conflict。本文将 x 拆成 $\text{xs_lo}[\text{blk_m}, 64]$ 与 $\text{xs_hi}[\text{blk_m}, 64]$ 两块独立缓冲，从布局上消除该冲突：

算法 3-3 · RoPE 内核

输入：查询张量 $X (M \times D)$, 旋转矩阵 cs 与 sn

输出：旋转后查询 Out

```

1 // 拆成两块独立缓冲，消除步长为 D/2 时的 2-way bank conflict
2 alloc xs_lo[blk_m, D/2], xs_hi[blk_m, D/2] in shared memory
3 load X[row_tile, 0 : D/2] -> xs_lo
4 load X[row_tile, D/2 : D ] -> xs_hi
5 for each (i, j) in parallel over [blk_m, D/2] do
6     c = cast(cs[i, j], fp32)
7     s = cast(sn[i, j], fp32)
8     x0 = cast(xs_lo[i, j], fp32)
9     x1 = cast(xs_hi[i, j], fp32)
10    Out[row_tile+i, j ] = cast(x0*c - x1*s, fp16)
11    Out[row_tile+i, j+D/2] = cast(x0*s + x1*c, fp16)

```

AutoTune 搜索维度为 $(\text{blk_m}, \text{threads})$ ，共 256 个 config。 M 较大时最优 blk_m 在 64~128； M 较小时较小的 blk_m 反而更优，因为更细的 Grid 能更充分利用 SM 并行度。

3.2.3 GQA Prefill 算子

3.2.3.1 实现结构

Prefill 内核 gqa_prefill_fwd 采用外循环按 Q 行、内循环扫描 K/V 的结构，Grid 为 $(\lceil S/\text{block_M} \rceil, H_q, B)$ ：

算法 3-4 · GQA Prefill 内核

输入：查询 Q, 键 K, 值 V (均为 $[B, S, H, D]$ 张量)

输出：注意力输出 Out [B, S, H_q, D]

```

1  Grid: (ceil(S/block_M), H_q, B); kv_h = head_id // gqa_group
2  alloc Q_shared[block_M, D], K_shared[block_N, D], V_shared[block_N, D] in SMEM
3  alloc acc_o[block_M, D], logsum[block_M], m[block_M] in register (fp32)
4  load Q[batch, q_tile, head, :] -> Q_shared
5  init: acc_o = 0, logsum = 0, m = -inf
6  // 两阶段异步流水：预取 K/V 与 GEMM 完全重叠
7  for kk = 0 to ceil(S/block_N) - 1 pipelined do
8      load K[batch, kk*block_N, kv_h, :] -> K_shared // async prefetch
9      mask causal positions to -inf in acc_s
10     acc_s = Q_shared * K_shared^T // QK^T GEMM, fp16->fp32
11     m_prev = m
12     m = max(m, row_max(acc_s)) // online max 更新
13     scores_scale = exp2(m_prev - m) * scale
14     acc_s = exp2(acc_s * scale - m) // softmax 分子
15     logsum = logsum * scores_scale + row_sum(acc_s) // 分母累积
16     acc_o = acc_o * scores_scale // 历史输出重缩放
17     load V[batch, kk*block_N, kv_h, :] -> V_shared // async prefetch
18     acc_o += cast(acc_s, fp16) * V_shared // PV GEMM
19 Out[batch, q_tile, head, :] = cast(acc_o / logsum, fp16)
    
```

T.Pipelined(loop_range, num_stages=num_stages) 声明 K/V 加载与 QK^T 计算的异步流水：在计算第 kk 块时，异步预取第 $kk + 1$ （乃至 $kk + 2$ ）块的 K/V。

3.2.3.2 规模因子与 exp2 技巧

exp 函数的硬件实现通常基于 $\exp_2$ 。将 softmax 的 $\exp(s_i/\sqrt{D})$ 改写为 $\exp_2(s_i \cdot \text{scale})$ ，其中 $\text{scale} = (\sqrt{D})^{-1} \cdot \log_2 e \approx 0.1275$ ($D=128$ 时)，可在开启 FAST_MATH 时映射到更快的 $\exp_2$ 硬件指令。累加用 FP32 避免 FP16 的溢出与精度损失，仅在最后输出时 cast 回 FP16。

3.2.3.3 GQA 头映射

与 Decode 类似，Prefill 通过 $\text{kv_h} = \text{by}$ // G 隐式完成 GQA 头映射，KV 张量保持 [B, S, H_{kv}, D] 原始布局，避免 repeat_interleave 带来的显存翻

倍。

AutoTune 搜索维度为 (block_M, block_N, num_stages, threads)，共 162 个 config。典型最优 config 为 (block_M=128, block_N=64, num_stages=2, threads=256)，长序列下有效算力可达 150+ TFLOPS。

3.2.3.4 微架构实证分析

GQA Prefill 的关键优化是 num_stages=2 的异步软件流水。表 3-2 给出 $B=2, S=4096$ (causal) 下 stages=1 与 stages=2 的 ncu 计数器对比。

表 3-2 GQA Prefill ncu 计数器对比

指标	stages=1	stages=2
Kernel 延迟 (ms)	4.921	4.424
Grid $\times$ Block	(32,64,2) $\times$ 256	
Tensor Core 利用率 (HMMA %)	11.8%	13.1%
SM 吞吐占峰值比例 (%)	45.7%	50.8%
SM 占用率 (Occupancy, %)	12.5%	12.5%
L2 命中率	94.7%	94.7%
DRAM 读取 (MB)	168.6	167.9
Warp 停顿原因 Top-3		
Wait 停顿（等待异步操作完成）	1.31	1.33
数学流水线限流（Math Pipe Throttle）	1.06	0.98
Long Scoreboard（等待 HBM）	0.93	0.12

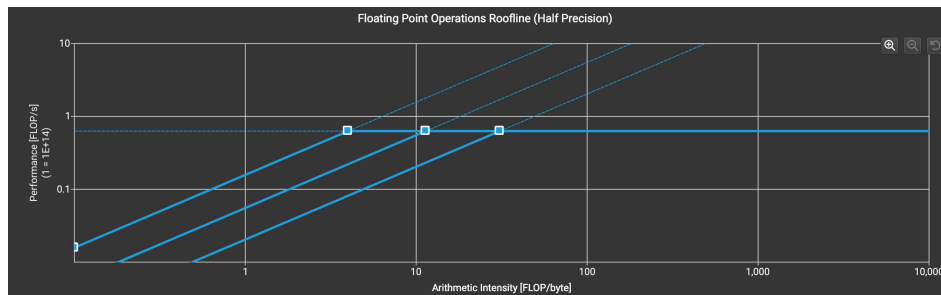
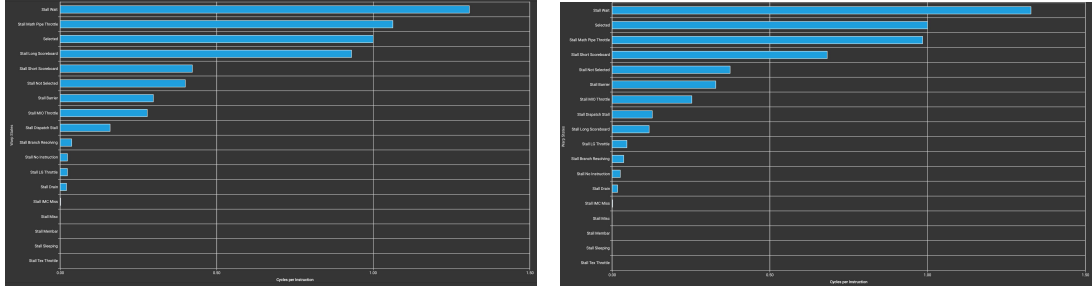


图 3-3 GQA Prefill Roofline

图 3-3 中，算子数据点落在 Roofline 平坦段 ($AI \approx 1820$ FLOP/Byte $\gg$ 屋脊点 156 FLOP/Byte)，表明 GQA Prefill 属于计算受限算子——在此区间 HBM 带宽已不是瓶颈，提升性能的关键在于提高 Tensor Core 利用率与减少计算停顿。

图 3-4 展示了两阶段配置下 warp 停顿来源的变化。stages=1 时 Long Scoreboard 排名第 4 (≈ 0.93 cycles/inst)，说明 warp 在等待 HBM 数据到达 SMEM 期间大量空转。开启 stages=2 后，cp.async 在当前 GEMM 计算期间预



(a) stages=1 (Long Scoreboard 为第 4 大停顿源) (b) stages=2 (Long Scoreboard 显著下降)

图 3-4 GQA Prefill Warp 停顿分布

取下一块 K/V，Long Scoreboard 降至第 10 位 (≈ 0.12 cycles/inst，降幅 87%)；与此同时数学流水线限流 (Math Pipe Throttle) 上升为次要瓶颈，说明 Tensor Core 已更充分地利用。

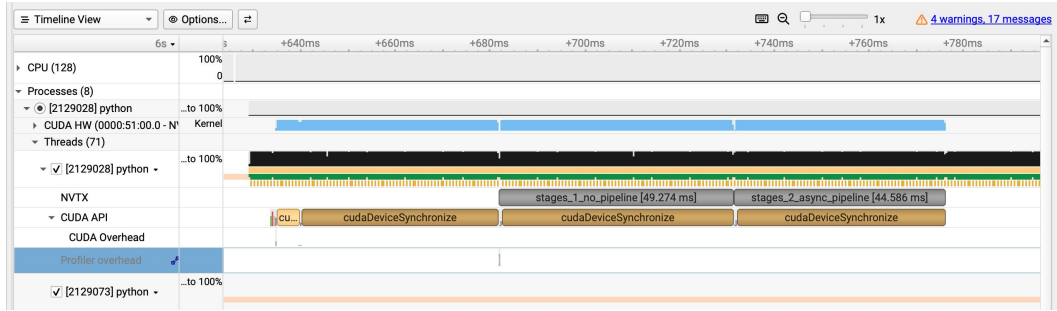


图 3-5 nsys 时间线对比

图 3-5 通过 NVIDIA Nsight Systems（系统级追踪工具，简称 nsys）的连续多次采集独立验证上述 ncu 测量：时间轴上以 NVTX 标记区分无流水线 (stages=1) 与异步流水线 (stages=2) 两段区间，每段内各执行 10 次 kernel launch 以累计区间耗时。NVTX 标记区间 stages_1_no_pipeline 总耗时 49.274 ms，stages_2_async_pipeline 总耗时 44.586 ms，性能提升 1.105 倍；折算单次延迟分别为 4.927 ms 与 4.459 ms，与 ncu 单次采集 (4.921 ms / 4.424 ms) 误差均在 0.5% 以内，确认了软件流水约 10% 的稳定加速效果。

3.2.4 GQA Decode 算子

3.2.4.1 Split-K 动机

Decode 时 $S_q=1$ ， QK^T 退化为 $[1 \times kv_len]$ 。若沿 Q 方向 tile，每个 tile 只有 1 行，Tensor Core $M=16$ 的约束完全无法利用。Split-K 将 KV 维度切为 num_split 段，每段独立计算 partial 注意力（含 partial softmax 统计量 m, ℓ ），最

后跨段做 reduction 合并^[28]。

相对单 pass 实现，Split-K 的代价是需要一次额外的 reduction kernel 合并 partial 输出。但收益是 Decode 路径可以利用 Tensor Core 的行并行度（跨 heads 或跨 splits）。

3.2.4.2 TileLang 实现

内核 gqa_flash_decode_kernel 的 Grid 为 $(B, H_q/\text{valid_block_H}, \text{num_split})$ ，每 block 处理一组 heads 与一段 KV：

算法 3-5 · GQA Decode 内核

输入：查询 Q, 键 K, 值 V (均为 $[B, 1, H, D]$ 张量)

输出：部分输出 PartialO, 对数求和指数 glse

```

1  // Split-K: KV 序列沿长度维划分为 num_split 段并行处理
2  Grid: (B, ceil(H/block_H), num_split); kv_seg = kv_len // num_split
3  alloc Q_shared[block_H, D], K_shared[block_N, D], V_shared[block_N, D] in SMEM
4  alloc acc_o[block_H, D], logsum[block_H], m[block_H] in register (fp32)
5  load Q[batch, 0, head_tile, :] -> Q_shared
6  init: acc_o = 0, logsum = 0, m = -inf
7  for kk = 0 to ceil(kv_seg/block_N) - 1 pipelined do
8      offset = split_id * kv_seg + kk * block_N
9      load K[batch, offset, kv_h, :] -> K_shared           // async prefetch
10     acc_s = Q_shared * K_shared^T                         // QK^T GEMM
11     m_prev = m
12     m = max(m, row_max(acc_s))                             // online max 更新
13     scores_scale = exp2(m_prev - m) * scale
14     acc_s = exp2(acc_s * scale - m)
15     logsum = logsum * scores_scale + row_sum(acc_s)
16     acc_o = acc_o * scores_scale
17     load V[batch, offset, kv_h, :] -> V_shared           // async prefetch
18     acc_o += cast(acc_s, fp16) * V_shared
19     PartialO[batch, split_id, head_tile, :] = acc_o
20     glse[batch, head_tile, split_id] = log2(logsum) + m * scale // 跨段 reduce 所需

```

最后一步通过一个小 kernel 跨 num_split 段做 log-sum-exp rescaling 与

partial 输出加权求和，得到最终输出。

Qwen3 配置下 Q 有 $H_q=64$ 头，KV 有 $H_{kv}=8$ 头，group size $G=8$ ；Kernel 通过 $\text{cur_kv_head} = \text{hid} // (\text{kv_group_num} // \text{valid_block_H})$ 将每个 Q head block 映射到对应 KV head，从而避免对 KV 张量做 repeat_interleave 物化。

AutoTune 搜索维度为 (block_N, block_H, num_split, num_stages, threads)，共 480 个 config。Decode 内核时间在 0.1 ms 量级，噪声占比较高，AutoTune 的重复测量次数因此设为 60（其他算子为 20）以确保搜索结果稳定。

3.2.4.3 微架构实证分析

针对 Split-K 方案，在 $B=16$ 、KV 序列长度 4096、 $S_q=1$ 下对 split=1 与 split=4 做 ncu 计数器对比，结果见表 3-3。

表 3-3 GQA Decode ncu 计数器对比

指标	split=1	split=4
Kernel 延迟 (μs)	302.3	229.7
Grid $\times$ Block	(16,8,1) $\times$ 128	(16,8,4) $\times$ 128
Grid 总 blocks	128	512
SM 占用率 (Occupancy, %)	6.27%	6.20%
SM 吞吐占峰值比例 (%)	22.6%	29.8%
Tensor Core 利用率 (HMMA %)	9.19%	7.97%
DRAM 读取 (MB)	269.9	270.8
L2 命中率	25.9%	24.6%
Warp 停顿原因 (居前项)		
Wait 停顿 (等待异步操作完成)	1.84	1.81
Long Scoreboard (等待 HBM)	0.68	0.95
Short Scoreboard (等待 SMEM)	0.40	0.32

该结果表明 Split-K 的主要收益来自更高的 grid 并行度 (128 $\rightarrow$ 512 blocks) 与更高 SM 吞吐，而不是占用率 (occupancy) 提升；其本质是以更多并发访存请求提高带宽利用。

3.2.4.4 分桶调度策略

TileLang kernel 的 tile 参数 (block_N、block_H、num_split、num_stages、threads，共 480 个配置点) 通过 @autotune 在首次调用时搜索最优配置，此后缓存复用。然而在端到端推理中，KV Cache 的序列长度 kv_len 随生成步数单调增长（从数十到数千），若每个新出现的形状都触发一次 autotune，会引入不可

接受的一次性延迟——480 个 config 逐一实测，单个形状约需 3–5 分钟，而生成过程中 kv_len 可取数百至数千个不同值，累计开销远超实际推理时间。

本文设计的 _bucketed_decode 策略以形状分桶绕开上述问题：在端到端微基准启动前，对代表性 kv_len 集合（如 {512, 1024, 2048, 4096, 8192}）离线运行 autotune，将最优 tile 写入 decode_configs.json；运行时每步 Decode 将实际 kv_len 映射到已探测集合中满足 $\text{bucket} \geq \text{kv_len}$ 的最小桶，若无更大桶则取最大桶并截断，必要时以零填充（F.pad）对齐 KV 形状；随后直接用该桶对应的预存参数实例化 kernel，全程不再触发 autotune，从而消除推理期的编译延迟。

分桶函数 pick_bucket_for_kv 的实现如下：

算法 3-6 · _bucketed_decode 实现

输入：查询 q, 键 k, 值 v 张量，可行桶列表 feasible_kvs

输出：解码后的注意力输出

```

1  function pick_bucket(actual_kv, buckets):
2      for b in sorted(buckets) do
3          if b >= actual_kv then return b
4      return buckets[-1]
5
6  function bucketed_decode(q, k, v):
7      kv_bucket = pick_bucket(k.seqlen, choices)
8      k = pad_or_slice(k, target_len=kv_bucket)
9      v = pad_or_slice(v, target_len=kv_bucket)
10     cfg = fixed_cfg[kv_bucket]           // 离线 AutoTune 最优参数
11     kernel = compile_kernel(kv_bucket, cfg)
12     return kernel(q, k, v)

```

本文以 128 token 为桶间距，即 $\text{bucket}(l) = \lceil l/128 \rceil \times 128$ 。每次 Decode 步的冗余 KV 读取量最多为 127 token。均匀分布假设下单步平均相对开销约为 $63.5/l$ ，随 l 增大迅速收敛至 0。对整条生成轨迹（ l 从初始 prompt 长度线性增长至最大值），因长尾步骤贡献权重更大，实际平均开销远低于上界。此外，零填充仅增加 KV 读取量，不影响 QKV 计算结果的正确性（mask 保证填充位的注意

力权重为 0)。各序列长度下的相对开销上界为

$$\delta_{\max}(l) = \frac{127}{l} \approx \begin{cases} 6.2\% & l = 2048 \\ 3.1\% & l = 4096 \\ 1.6\% & l = 8192 \end{cases} \quad (3-4)$$

以最大 kv_len= 4096 为例，无分桶时生成过程中每遇新形状即 autotune，至多 4096 次探测，总时长约 $4096 \times 3\text{--}5 \text{ min} \approx 8\text{--}14$ 天。若采用粗粒度四档 {512, 1024, 2048, 4096}，离线代价仅 $4 \times 3\text{--}5 \text{ min} = 12\text{--}20$ 分钟，但相邻档最大间距 2048 token，首档内冗余可达 $2047/2048 \approx 100\%$ 。本文采用的 128 token 粒度共 32 档，离线约 1.6–2.7 小时，将冗余上界压在 127/128（主要影响最短档首步）；在 $l \geq 1024$ 的典型区间内冗余通常可降至 12% 以下，从而在离线 tune 成本与线上性能损失之间取得较稳妥的折中。

综合来看，128 token 粒度将离线探测压缩到数小时量级（相对无分桶的数日），并把中长序列（ $l \geq 2048$ ）的冗余控制在约 6% 以下，工程上可接受。

3.2.5 SwiGLU 算子

3.2.5.1 融合动机

Qwen3 FFN 公式：

$$\text{FFN}(x) = W_2 \cdot \text{SwiGLU}(W_1 x, W_3 x), \quad \text{SwiGLU}(a, b) = \text{silu}(a) \odot b \quad (3-5)$$

其中 $W_1 x, W_3 x \in \mathbb{R}^{M \times d_{\text{fm}}}$ ， $d_{\text{fm}}=27648$ ， $M = B \cdot S$ 。

PyTorch eager 将 $\text{F.silu}(\text{gate}) * \text{up}$ 拆成两个 kernel，先 $\text{silu}(\text{gate})$ 写出中间张量 tmp（读 gate、写 tmp），再 $\text{tmp} * \text{up}$ 得到输出（读 tmp、读 up、写 out）。HBM 总流量约为 $5MN_{\text{fm}} \cdot 2$ 字节（每元素 3 读 + 2 写）；融合实现合并为单 kernel，仅读 gate、读 up、写 out，流量降至 $3MN_{\text{fm}} \cdot 2$ ，节省约 40%。

3.2.5.2 TileLang 实现

内核 swiglu_fwd 为 1D Grid $\lceil M/\text{blk}_m \rceil$ ，每 block 沿 N 维循环 blk_n 步长：

算法 3-7 · SwiGLU 内核

输入：门控投影 Gate ($M \times N$), 上升投影 Up ($M \times N$)

输出：融合输出 Out ($M \times N$)

```

1  Grid: ceil(M/blk_m);  alloc gf[blk_m, blk_n], uf[blk_m, blk_n] in register (fp32)
2  for kk = 0 to ceil(N/blk_n) - 1 do
3      load Gate[row_tile, kk*blk_n : (kk+1)*blk_n] -> gf
4      load Up  [row_tile, kk*blk_n : (kk+1)*blk_n] -> uf
5      // silu(gate) * up 在寄存器内完成，无中间张量写回 HBM
6      for each (i, j) in parallel do
7          silu_g = gf[i,j] / (1 + exp(-gf[i,j]))
8          Out[row_tile+i, kk*blk_n+j] = cast(silu_g * uf[i,j], fp16)

```

启用 TL_ENABLE_FAST_MATH, T.exp 映射为 __expf, 约 5% 提速。

AutoTune 搜索维度为 (blk_m, blk_n, threads), 共 288 个 config。对 $N=27648$, 最优 config 在 (blk_m=16, blk_n=2048, threads=512) 附近。

3.2.6 W4A16 GEMV 算子

3.2.6.1 动机与问题定义

五类基础算子均在 FP16 精度下运行。然而对于 Decode 阶段, MLP 层的三个全连接投影 (gate、up、down proj) 是主要的显存带宽消耗点: 以 Qwen3-32B 为例, 单层 MLP 的三个权重矩阵合计约 849 MB (FP16), 每生成一个 token 均需从 HBM 完整读取一遍, 与有效计算量之比 (算术强度) 仅约 1 FLOP/byte, 远低于 A100 的 roof 拐点 (约 $312 \text{ TFLOP}/2 \text{ TB/s} = 156 \text{ FLOP/byte}$)。

W4A16 (INT4 权重 + FP16 激活) 将权重存储位宽从 16 位压缩至 4 位, HBM 权重读取量降至原来的 $\frac{1}{4}$, 从而在不改变模型精度规格 (激活仍为 FP16) 的前提下将 MLP Decode 的访存压力降低约 $4\times$ 。本节介绍权重预处理流程 (packed 到 LOP3 interleaved 布局) 与内核的微架构分析; Decode 阶段的分桶调度策略已在 3.2.4 节统一描述, monkey-patch 替换机制见 3.3 节。

在量化粒度的选型上, W8A8 (权重与激活均量化至 INT8) 与 W4A16 (权重压缩至 INT4、激活保持 FP16) 是大语言模型推理中最常见的两条路线, 两者

在 Decode 场景下的性能特征存在本质差异。

Decode 阶段每步仅生成一个 Token, $S_q = 1$, MLP 的全连接投影退化为矩阵向量乘 (GEMV)。设权重矩阵 $W \in \mathbb{R}^{N \times K}$, 输入激活 $x \in \mathbb{R}^K$, GEMV 的算术强度为:

$$AI_{GEMV} = \frac{2NK}{NK \cdot b_w/8 + K \cdot b_a/8 + N \cdot b_a/8} \approx \frac{16}{b_w} \quad (3-6)$$

其中 b_w 与 b_a 分别为权重与激活的位宽 (bit)。代入 W8A8 ($b_w=8$) 得 $AI \approx 2$ FLOP/byte; 代入 W4A16 ($b_w=4$) 得 $AI \approx 4$ FLOP/byte。两者均远低于 A100 屋脊点 156 FLOP/byte, 均为访存受限算子——此时决定延迟的不是计算峰值, 而是 HBM 带宽与权重数据量的乘积。

在访存受限区, 延迟近似正比于读取的字节数。W4A16 权重位宽为 FP16 的 $\frac{1}{4}$, 权重读取量最小; W8A8 权重位宽是 W4A16 的 $2\times$, 延迟理论上高出约 $2\times$ (激活读取量两者接近, 可忽略差异)。此外, W8A8 需要对激活动态量化 (运行时对每个 Token 计算尺度因子), 引入额外的逐 Token 归约操作; 而 W4A16 保持 FP16 激活, 省去动态量化开销, 也避免了激活量化在长尾分布下的精度退化。综合以上分析, W4A16 在 Decode 阶段同时具备最低的 HBM 带宽压力与最简的运行路径, 是本系统的首选量化方案。

3.2.6.2 内核架构: INT4 反量化与 FP16 累加

W4A16 GEMV 的核心挑战在于 GPU 缺乏 4 位原生类型, INT4 权重须在运算前解包为可参与乘加的 FP16; 朴素路径 (移位、掩码、整型与浮点转换) 需 5~6 条指令, 在 GEMV 热路径上代价显著。本文内核以三段流水将解包延迟藏入访存与计算重叠之中。首先, 权重按“两枚 INT4 打包为一字节 INT8”存放, 低 nibble 存偶数列、高 nibble 存奇数列, 使 HBM 占用恰为 FP16 的 $\frac{1}{4}$, 单次 DRAM 事务可搬运更多有效参数; T.Pipelined 配合 `cp.async` 在计算当前分块时预取下一分块至 Shared Memory, 以掩盖 HBM 延迟。随后, LOP3 (`lop3.b32`) 在单条指令内对三枚 32 位寄存器施加任意布尔组合 (由 `immLut` 指定), 结合 FP16 的 IEEE 754 位型与偏置魔数 C_{bias} , 将 nibble 直接映射到尾数位, 相当于近乎零开销的 INT4→FP16 重解释, 再辅以一条 FP16 减法完成零点校正, 全程约 4 条 PTX, 较朴素路径省约 60% 解码指令; `interleave_weight()` 则预先重排 nibble, 使

lop3.b32 可无分支取数。最后，解包得到的 FP16 权重与激活 x 做向量内积，累加保留在 FP32，再乘行尺度 s_i （广播至 Shared Memory）写回 HBM。

上述三阶段在 TileLang 中的实现如下：

算法 3-8 · W4A16 GEMV 内核

输入：激活 A ($M \times K$, FP16), 量化权重 B ($N \times K/2$, INT4 packed)

输出：输出 C ($M \times N$, FP16)

```

1  // 线程块: threadIdx.x = reduce_thread (K 向规约), threadIdx.y = n_partition (N
    向分组)
2  Grid: (ceil(N/n_partition), M)
3  alloc A_local[8] (fp16), B_q[4] (int8), B_dq[8] (fp16), acc (fp16) in register
4  init acc = 0
5  for ko = 0 to ceil(K/block_K) - 1 do
6      vectorized load A[row, ko*block_K + kr*8 : +8]    -> A_local  // 8xFP16
7      vectorized load B[col, ko*block_K/2 + kr*4 : +4]  -> B_q      // 4xINT8 (8
        nibbles)
8      // LOP3 位运算: 单指令将 INT4 nibble 映射至 FP16 尾数, 完成反量化
9      B_dq = lop3_dequant(B_q)                          // INT4 -> FP16, ~4 PTX 指令
10     for ki = 0 to 7 do
11         acc += A_local[ki] * B_dq[ki]
12     warp_reduce_sum(acc, dim=kr)                       // warp 内跨 reduce_thread
        求和
13     if kr == 0 then
14         C[row, col] = acc

```

内核以 `reduce_thread=32`（一个 warp）负责 K 维度的分块归约，`n_partition=4` 将 N 维度进一步分配到 `threadIdx.y`，每个线程迭代 K/block_K 轮：每轮向量化加载 8 个 FP16 激活与 4 个 INT8（含 8 个 INT4 nibble），通过 `T.call_extern` 调用 LOP3 PTX 内联函数完成单指令反量化，随后执行 FP16 向量内积累加。循环结束后通过 warp 级归约合并跨线程的部分和。

3.2.6.3 权重预处理：packed 到 LOP3 interleaved 布局

TileLang 的 W4A16 GEMV kernel 使用 LOP3 位运算指令（lop3.b32）在寄存器中即时反量化 INT4 权重至 FP16，无需中间存储。LOP3 指令的正确执行要

求权重在内存中按特定的 interleaved 顺序排列，这一顺序与常规 INT4 打包格式（低 nibble 存偶数列、高 nibble 存奇数列）不同，因此在 patch 时需依次完成布局转换。首先在 CPU 上将 FP16/BF16 权重矩阵 $W \in \mathbb{R}^{N \times K}$ 做逐行对称 INT4 量化（range $[-8, 7]$ ），得缩放因子 $s_i = \max_j |W_{ij}|/7$ 与量化值 $\hat{W}_{ij} = \text{round}(W_{ij}/s_i)$ 。随后将相邻两列并入一个 int8 字节（低 nibble 对应偶数列，高 nibble 对应奇数列），得到形状 $[N, K/2]$ 的 packed 张量。最后调用 `interleave_weight(packed, nbits=4, target_dtype="float16")` (`tilelang.quantize.utils`) 按 LOP3 访存模式重排 nibble，输出形状仍为 $[N, K/2]$ int8，而内部布局已与 CUDA 寄存器文件对齐。

为节省 GPU 显存，上述量化步骤全部在 CPU 上执行（`_quant_pack_cpu` 函数），只将最终 interleaved int8 张量发回 GPU，原始 FP16 权重可在 patch 完成后从 GPU 显存释放。Interleave 变换对每个权重矩阵仅执行一次（在 patch 阶段），之后每次 Decode 步均直接使用预处理后的 `_w4_gate`、`_w4_up`、`_w4_down` 张量，无额外开销。

3.2.6.4 微架构实证分析

形状取 $M=4$, $N=25600$, $K=5120$ （Gate 投影高层，与第四章表 4-8 中 Gate proj 的形状参数一致）。采集时以 `ncu -k` 仅统计 TileLang 的 `main_kernel` 与 Triton 的 `_w4a16_gemv_kernel`，排除权重预处理阶段的辅助 kernel；各后端连续 3 次 launch 取中位数，硬件为 A100。

表 3-4 W4A16 GEMV ncu 内核指标

指标	TileLang	Triton
Kernel 延迟 (μs)	165.3	516.8
Kernel 名 (demangled)	<code>main_kernel</code>	<code>_w4a16_gemv_kernel</code>
Grid $\times$ Block (1D 口径)	25600×128	6400×128
Registers / thread	38	54
SM 占用率 Occupancy (%)	69.1	53.9
Waves per SM	19.8	6.58

表 3-4 表明：在相同权重形状下，TileLang 内核相对 Triton 性能提升约 3.1 倍；两者每 block 均为 128 线程，但 Grid 规模与每线程寄存器数不同（Triton 寄存器压力更高、占用率更低），对应不同的并行划分与解码/访存组织。

表 3-5 与表 3-4 对照可见：TileLang 的 DRAM 吞吐达到峰值的 83.5%、L2 吞

表 3-5 W4A16 GEMV ncu 存储吞吐

指标 (SOL Throughput)	TileLang	Triton
L2 Cache	78.8	12.2
DRAM	83.5	8.0

吐达到 78.8%，Triton 分别仅为 8.0% 与 12.2%。W4A16 GEMV 为流式权重读取场景，每个权重仅访问一次，L2 缓存无法有效复用，DRAM 带宽利用率直接决定性能；Triton 的 L2 吞吐略高于其 DRAM 吞吐，说明部分流量确实命中了 L2，但整体吞吐仍极低，根本原因在于缺乏显式流水导致 SM 空转。TileLang 通过 T.Pipelined 与 cp.async 在计算当前分块时异步预取下一分块，内存控制器持续有飞行中请求，DRAM 与 L2 带宽均得到充分利用，与表中更短的 wall-clock 内核时间一致。

3.2.7 代码复杂度对比

图 3-6 对比了六类算子 TileLang 与 Triton 实现的代码行数（不含基准测试函数与纯注释行）。RoPE 上 Triton 需显式传递 M/S/H/stride_xm/stride_xd 等并在 kernel 内手工求 position，TileLang 则借 T.copy 隐去 stride，把 position 计算嵌入 T.Parallel，核心逻辑约 20 行，相对 Triton（248 行）代码量压缩约 3.5 倍。

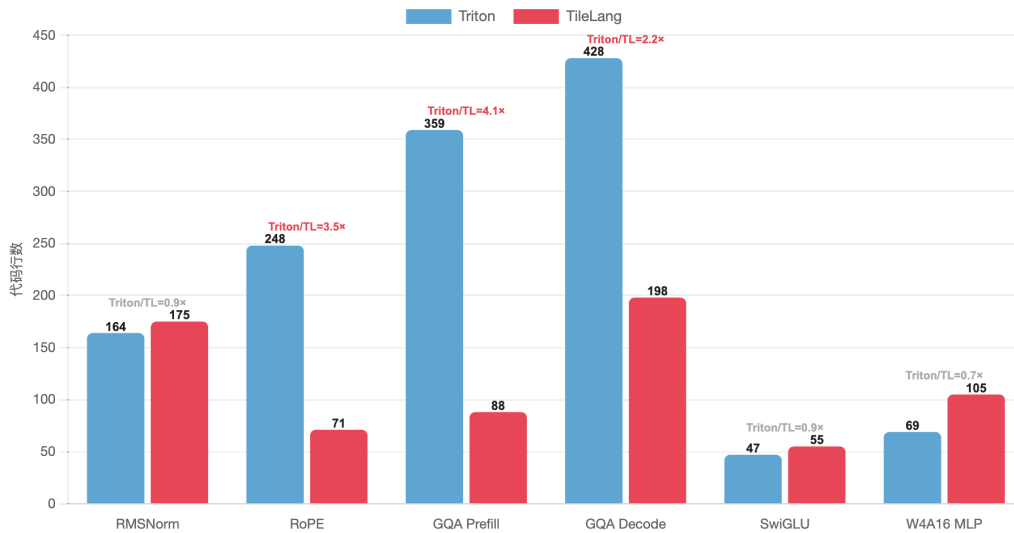


图 3-6 六算子代码行数对比

GQA Prefill 中 Triton 须手写 group 映射、softmax rescale 与多轮 t1.load，TileLang 用 T.gemm/T.reduce_max/T.Pipelined 表达同一计算图并由编译器补全 layout 与流水，行数由 359 降至 88（约 4.1x）。

GQA Decode 的 Triton 路径要手工管 Split-K 的 KV tile、跨 CTA softmax 与合并，TileLang 以 `T.Parallel/T.reduce` 描述 Split-K 并由编译器生成跨 CTA 归约，约 198 行对 428 行（约 $2.2\times$ ）。

RMSNorm 与 SwiGLU 属于简单归约或逐元素 (elementwise) 算子，两类 DSL 抽象收益有限，规模接近。W4A16 GEMV 出现少见反转：TileLang（105 行）多于 Triton（69 行），因 LOP3 反量化要求特定 interleaved 布局，TileLang 在 `T.copy` 中嵌入 nibble 重排与打包（约 30 行），而 Triton 可用 CUTLASS 风格位操作宏折叠为少量内联汇编；尽管如此，TileLang 的微基准仍快 $2\text{--}7.7\times$ ，说明行数与执行效率并无简单对应，为性能付出 interleave 逻辑仍值得。

3.3 推理调度引擎

推理调度引擎是系统的中间层，负责将算子内核层的高性能实现注入推理流程，并统一管理分词、KV Cache 与前向执行。图 3-7 以一个 Transformer Block（变换器层）为单元，完整呈现了推理数据流中各算子的计算职责、张量规格与替换状态。

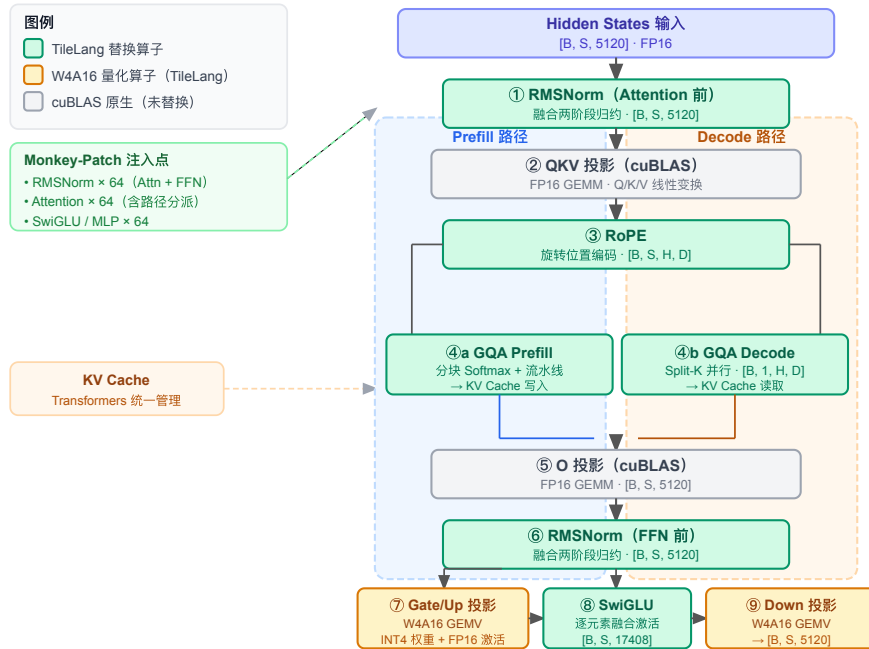


图 3-7 Qwen3-32B Transformer Block 推理数据流与算子替换总览

推理调度引擎通过运行时算子替换机制（monkey-patch）将 TileLang 高性能内核无侵入地注入 Qwen3-32B 的推理流程。具体做法是：在模型加载完成后，引

擎遍历所有模块，对 Qwen3RMSNorm、Qwen3Attention、Qwen3MLP 三类模块调用 `types.MethodType`，将原始的 `forward` 方法替换为 TileLang 包装实现，同时保留模块的权重参数与配置不变。替换在模型推理开始前一次性完成，稳态执行路径与原始 HuggingFace 实现完全等价，上层调用代码无需任何改动。

替换后的关键调度逻辑发生在 Attention 模块内部。由于 Prefill 与 Decode 两个阶段的计算形态截然不同，引擎在每次 Attention 的 `forward` 调用时，根据输入序列长度 S_q 实时分派内核： $S_q > 1$ 时调用 GQA Prefill 路径，走分块 Softmax 与流水线优化； $S_q = 1$ 时切换到 GQA Decode 路径，走 Split-K 并行规约。这一分派对 MLP 与 RMSNorm 的替换则是无条件的——每次 `forward` 均调用对应的 TileLang 内核，其中 MLP 的 Gate/Up/Down 投影采用 W4A16 量化内核，以 INT4 权重存储换取 Decode 阶段的带宽节省。如图 3-7 所示，九个算子中有七个被替换，仅 QKV 投影与 O 投影因 cuBLAS 已接近理论峰值而保留原始实现。

本节从替换契约、路径分派与可靠性机制三方面进一步说明该机制的设计。

3.3.1 替换契约

monkey-patch 的前提是 TileLang 算子在行为上与原始 PyTorch 算子完全等价，替换后上层代码无感知。为此，每个包装函数须在形状、数值与生命周期三方面与原始契约一致。形状与布局上，输入输出的形状、步幅(stride)、内存布局须与原始算子一致：RMSNorm 接受 $[M, N]$ 或 $[\dots, N]$ ，RoPE 接受 $[B, S, H, D]$ ，GQA Prefill/Decode 接受 $[B, S, H, D]$ 的 Q 与 $[B, S_{kv}, H_{kv}, D]$ 的 KV。数值上，输出相对 PyTorch 参考实现须通过 `allclose(atol=1e-2, rtol=1e-3)`，新算子接入前均运行 `verify_*`，未通过则不替换；FP16 下采用 $\text{atol} = 10^{-2}, \text{rtol} = 10^{-3}$ 的容差设置。生命周期上，JIT 与 AutoTune 仅在首次调用时触发，结果以输入 shape 为 key 缓存，稳态路径重复命中缓存，不重复编译，额外开销可忽略。

3.3.2 Prefill/Decode 路径分派

GQA 在两个阶段的计算形态截然不同，需要独立的内核实现。Prefill 时 $S_q = S_{kv} = N$ ， $QK^\top$ 是完整的矩阵乘，内核以分块 Softmax 处理，tile 参数以 $(\text{block_M}, \text{block_N}, \text{num_stages})$ 为主。Decode 时 $S_q = 1$ ， $QK^\top$ 退化为向量-矩阵乘，沿 Q 方向 tile 每块只有 1 行，Tensor Core 形同虚设。因此

Decode 改为沿 KV 维度切分（Split-K）：将 kv_len 拆为 num_split 段分别计算 partial 注意力与 softmax 统计量，再做一次 reduction 合并，tile 参数变为 $(block_N, block_H, num_split, num_stages)$ 。

两条路径在代码上完全独立，引擎在运行时根据 S_q 自动分派，对调用方透明。

3.3.3 系统可靠性保障

内核 JIT 编译失败时自动回退到 PyTorch 原生实现，推理链路不中断。通过环境变量 `TILELANG_VERIFY=1` 可开启数值验证，将内核输出与 PyTorch 参考实现做 `allclose` 比较，性能测试时关闭。关键内核前后插入 CUDA 事件（CUDA Event）记录耗时，输出 CSV 供微基准分析。

3.4 交互展现层设计

3.4.1 通用后端接口与多场景复用

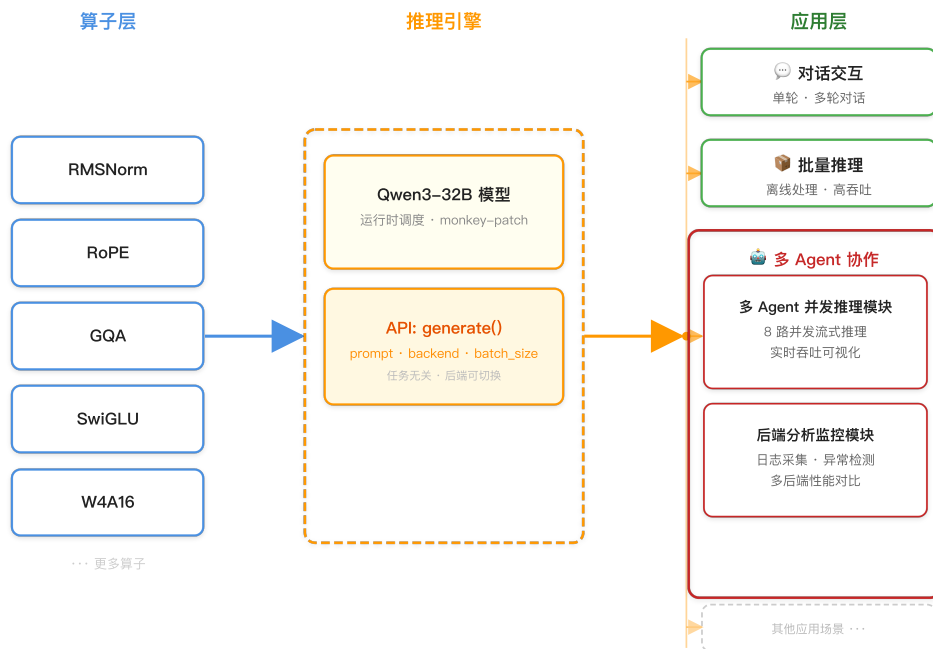


图 3-8 通用后端接口与多应用复用架构

3.2 节和 3.3 节构建了算子内核与推理调度引擎，两者对外统一暴露一个调用接口：

```
generate(prompt, backend, batch_size) -> token stream
```

prompt 是输入文本，backend 指定底层实现（TileLang / Triton / PyTorch），batch_size 控制并发数量。这个接口不包含任何与特定应用相关的逻辑，因此对话交互、离线批处理、多 Agent 协作等不同上层应用均可直接复用同一套推理后端（图 3-8），切换后端只需改变 backend 参数，无需修改应用代码。

在此接口之上构建应用时，不同场景之间存在若干共同的设计逻辑。第一，任务上下文独立维护。无论是单轮对话还是多 Agent 并发，每个任务都持有各自的输入上下文，推理侧为各任务维护独立的 KV Cache，互不干扰。第二，流式输出与异步回调。推理结果以逐 token 的方式流式返回，前端通过异步回调逐步更新，而不是等到生成完成后一次性呈现，这对用户感知的响应速度有直接影响。第三，性能指标伴随采集。吞吐、延迟等性能数据与推理过程同步采集，在不中断服务的情况下随时可观察系统状态。这三条逻辑与具体应用类型无关，是交互展现层在任何场景下都适用的基本实现约定。

3.4.2 多 Agent 场景与系统功能划分

本文以多 Agent 协作作为交互展现层的具体实现场景。在这一场景中，每个 Agent 承担不同的分析任务，持有各自独立的 prompt——例如，设计思路分析 Agent 的提示词聚焦于架构决策，性能分析 Agent 的提示词包含算子耗时数据，量化误差 Agent 关注数值精度，参数调优 Agent 针对超参搜索。各 Agent 携带不同的 System 指令与代码上下文，同时向后端发起并发推理请求，后端需并行处理这些各不相同的输入并逐 token 流式返回结果。为此，系统实现了两个面向不同使用方的功能模块。多 Agent 并发推理模块面向普通用户侧，用户通过该模块发起并发推理任务并实时查看各 Agent 的生成结果；后端分析监控模块面向服务提供商的服务器后台，运维人员通过该模块观察推理服务的运行状态、采集性能指标并对比不同后端实现的表现。两个模块共用同一推理后端实例，职责边界清晰。

3.4.3 多 Agent 并发推理模块

多 Agent 并发推理模块面向普通用户，核心设计目标是让用户能够一次性提交多个不同分析任务并实时获得各路结果。模块将输入拆分为三个独立部分：代码上下文、用户问题与各 Agent 的任务指令，三者提交时自动组合为各 Agent

各自的完整 prompt，使用户只需填写一份代码上下文，即可为设计思路、性能分析、量化误差、参数调优等不同任务分别配置专属指令，无需重复粘贴。

在输出侧，模块采用异步流式回调机制，每生成一个 token 即实时推送至对应 Agent 的展示卡片，用户无需等待全部任务完成便可同步浏览各 Agent 的推理进度，各卡片同步记录并展示每个 Agent 的生成耗时，用户可据此了解当前后端在并发负载下的实际响应速度。模块界面如图 3-9 所示。

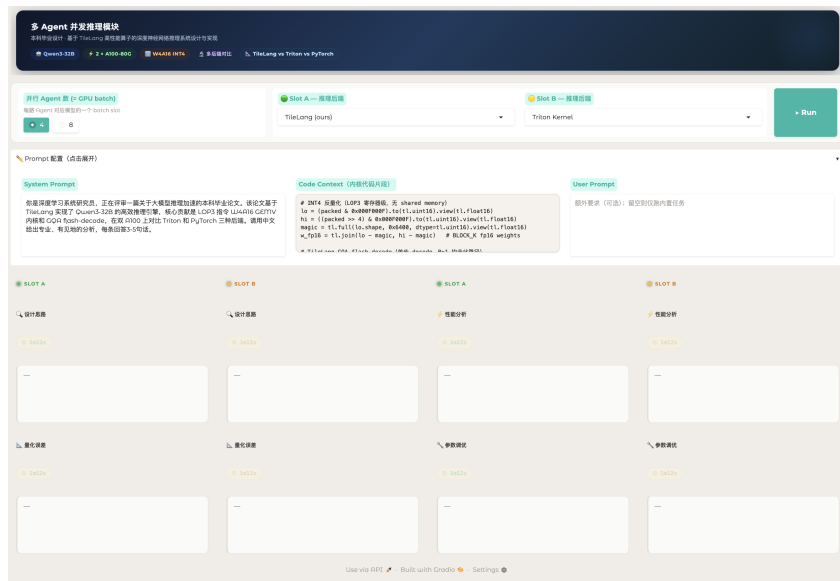


图 3-9 多 Agent 并发推理模块

3.4.4 后端分析监控模块

后端分析监控模块面向服务提供商的管理人员，核心设计目标是让管理人员无需查看底层代码即可掌握推理服务的运行健康状况与性能水平。模块以轮询方式持续从推理服务端拉取运行日志，覆盖算子调度、内核编译、显存分配等关键路径，使任何运行异常都能在管理界面上第一时间浮现，而不必登录服务器翻查日志。

在性能可视化上，模块在每次推理调用前后自动记录墙上时间并计算 Decode 吞吐（tokens/s），以面板形式实时呈现，管理人员可随时了解当前负载下各算子与推理后端的实际性能表现。

两个功能模块共用同一推理后端实例，通过统一接口 `generate(prompt, backend, batch_size)` 驱动，切换底层实现只需修改 `backend` 参数。这种设计使得对比 TileLang、Triton、PyTorch 三种后端在相同负载下的吞吐与延迟差异成

为一键操作，无需修改任何应用层代码。模块界面如图 3-10 所示。

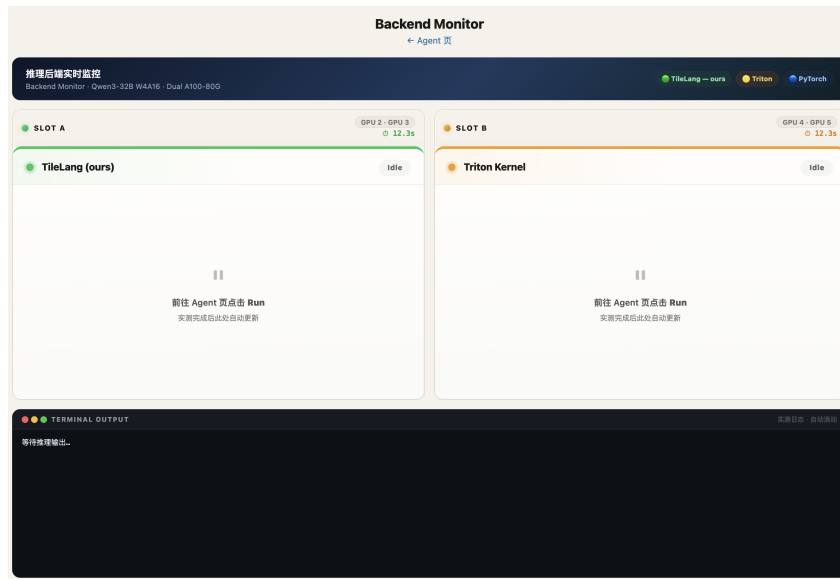


图 3-10 后端分析监控模块

3.5 本章小结

本章完成了面向多 Agent 推理场景的完整系统设计与实现。算子内核层针对六类热点算子的访存与计算瓶颈实施定向优化；推理调度层通过 monkey-patch 机制实现无侵入式算子替换，并对外暴露与应用场景无关的统一调用接口；交互展现层以多 Agent 协作为具体场景，在该接口之上实现了多 Agent 并发推理模块与后端分析监控模块，分别支撑多路并发推理的前端交互与不同后端实现的性能对比验证。

4 实验与评估

4.1 实验环境与方法论

4.1.1 硬件与软件栈

所有实验在下述固定栈上完成。

表 4-1 实验环境

项目	配置
GPU	NVIDIA A100 SXM4-80GB (SM_80, 312 TFLOPS BF16, 2.0 TB/s HBM)
稳定 SM 时钟	1410 MHz (热身后)
驱动 / CUDA	NVIDIA Driver 550.x / CUDA 12.1
Python	CPython 3.12
PyTorch	2.7.0+cu126
Triton	3.2.0
TileLang	0.1.8 dev (基于 TVM 0.17)
测试精度	FP16 (权重/激活), FP32 (中间累加)
数值容差	$\text{atol} = 10^{-2}$, $\text{rtol} = 10^{-3}$

4.1.2 测量协议

所有微基准均经 GPU 热身后取多次运行的中位数延迟，AutoTune 搜索开销不计入算子延迟。所有 TileLang 内核均与 PyTorch 参考实现通过数值验证后方参与对比。

4.1.3 测试形状

本文选取 4 个覆盖 Qwen3 推理中典型场景的 case（表4-2）。除另行注明外，下文微基准形状均默认 $H_q=64$, $H_{kv}=8$, $D=128$, $d_{\text{model}}=5120$, $d_{\text{ffn}}=27648$ ，与表 2-1 一致。

表 4-2 测试用例与形状

Case	批量/序列	Decode kv_len	场景
1	$B=2, S=4096$	—	长文本 prefill (双请求)
2	$B=4, S=2048$	8192	中等批量在线推理
3	$B=16, S=512$	4096	高吞吐 serving (大批量短序列)
4	$B=8, S=2048$	4096, 8192	大批量 prefill+decode

各算子的具体形状派生自上述 case：RMSNorm 与 SwiGLU 使用 $M = B \cdot S$ 行；RoPE 使用 $M = B \cdot S \cdot H$ 个 token；GQA Prefill/Decode 使用上述批量与 seq 组合。

4.2 算子微基准评测

本节对六类算子分别在不同输入规模下进行微基准测试，比较 TileLang、Triton 与 PyTorch 三种实现的延迟与吞吐，并附算子综合对比。

4.2.1 RMSNorm 微基准

4.2.1.1 结果

表 4-3 报告延迟中位数 (ms)；输入 $[M, N]$ 中 $N=5120$ ，与 Qwen3 隐藏维一致， M 由表 4-2 的 case 派生。

表 4-3 RMSNorm 微基准延迟

案例	M	PyTorch (ms)	Triton (ms)	TileLang (ms)
1	8192	1.248	0.175	0.145
2	8192	1.248	0.180	0.131
3	8192	1.248	0.174	0.128
4	16384	2.230	0.314	0.234

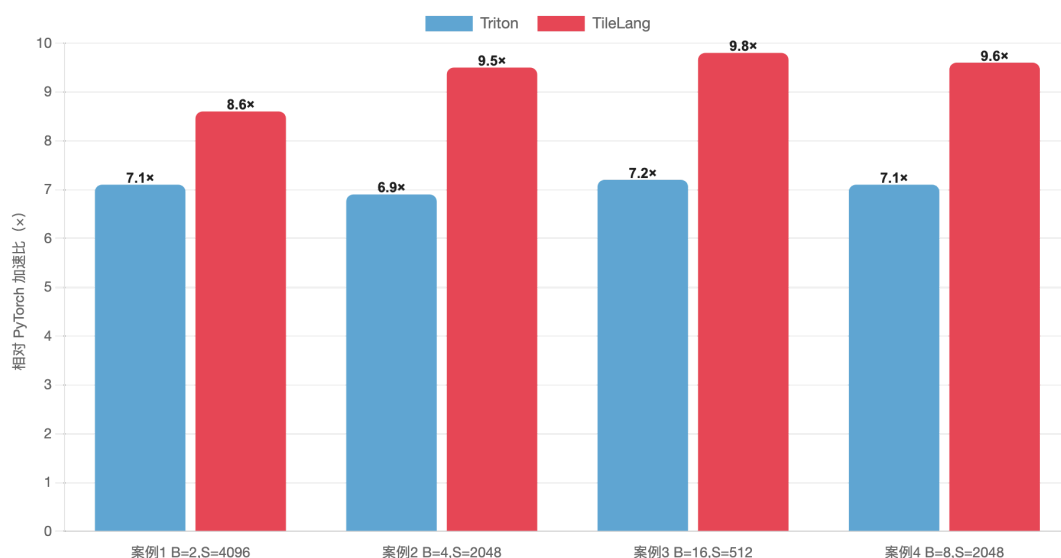


图 4-1 RMSNorm 相对 PyTorch 加速比

4.2.1.2 分析

与 Triton 相比, TileLang 在四个用例上均更快, 性能提升分别为 1.20、1.37、1.36、1.34 倍 (见图 4-7), 主要得益于两阶段片上 `inv_std` 复用及 `T.Pipelined` 对阶段二 K-tile 的异步预取; Triton 行归约虽在寄存器间传递 `inv_std`, 却未像 TileLang 那样显式流水化第二阶段的 HBM 读。相对 PyTorch, `F.rms_norm` 延迟约为 TileLang 的 9 至 10 倍, 除框架调度与启动开销外, `aten` 路径将求 RMS 与乘权重拆成两个 kernel, 而 TileLang 在单 kernel 内融合完成。按 Roofline 估算, 案例 3 ($M=8192, N=5120$) 理论 HBM 流量约 252 MB, 2.0 TB/s 对应时间下界约 0.126 ms, TileLang 实测 0.128 ms, 带宽利用率约 98% (部分命中 L2 使实访存略低于账面对应值)。

4.2.2 RoPE 微基准

4.2.2.1 结果

表 4-4 为 `rotate_half` 路径延迟中位数 (ms); 头维度 $D=128$, 行数 $M=B \cdot S \cdot H_q$, 由表 4-2 与模型头数派生。

表 4-4 RoPE 微基准延迟

案例	M	PyTorch (ms)	Triton (ms)	TileLang (ms)
1	524K	2.311	0.250	0.174
2	524K	2.517	0.248	0.174
3	524K	2.283	0.251	0.173
4	1024K	4.517	0.477	0.328

4.2.2.2 分析

TileLang 在全部 4 个测试用例上均快于 Triton, 加速比为 $1.44\times$ 、 $1.42\times$ 、 $1.45\times$ 、 $1.46\times$ (见图 4-7)。加速来源主要是 AutoTune 在本轮搜索中选出了更激进的 `blk_m` 配置, 使共享内存 `split-buffer (xs_lo, xs_hi)` 的 bank conflict 消除效果得到充分发挥。

案例 1–3 的 M 完全相同 (均为 524288), 三者延迟在 0.173~0.174 ms 之间, 差异 $<1\%$, 说明当前 AutoTune 配置下结果高度稳定。案例 4 ($M=1024K$) 翻倍, 延迟 0.328 ms, 接近线性扩展。

PyTorch 的 `apply_rope_rotate_half` 走 `torch.cat` 加逐元素乘加, 多次分

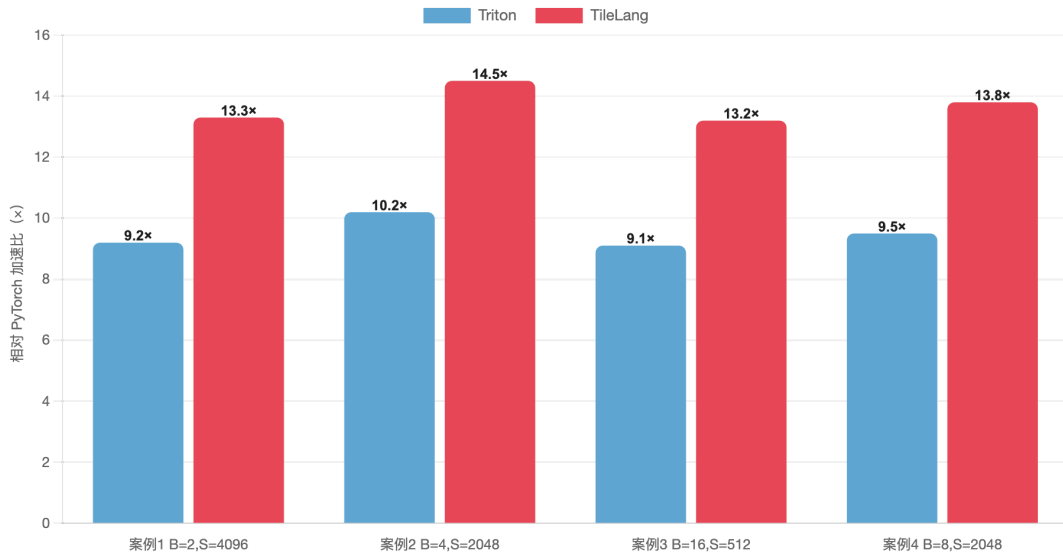


图 4-2 RoPE 相对 PyTorch 加速比

配中间张量；TileLang 单内核融合后相对 PyTorch 性能提升约 13 至 15 倍。

4.2.3 GQA Prefill 微基准

4.2.3.1 结果

表 4-5 为因果注意力 (causal) 下延迟中位数 (ms), $H_q=64, H_{kv}=8, D=128$ 。

表 4-5 GQA Prefill 微基准延迟

案例	$B \times S$	PyTorch (ms)	Triton (ms)	TileLang (ms)
1	2×4096	46.18	11.06	3.59
2	4×2048	24.13	5.81	1.98
3	16×512	9.27	1.86	0.81
4	8×2048	48.16	11.37	3.89

4.2.3.2 分析

TileLang 在全部 4 个测试用例上显著超过 Triton，性能提升分别为 3.08、2.93、2.30、2.92 倍（见图 4-7）。有效算力：案例 1（批大小 2，序列长度 4096）达到 153 TFLOPS，占 A100 BF16 理论算力 312 TFLOPS 的 49%，这已是非 cuBLAS 手写路径的较高水平。

PyTorch 原生路径 (matmul + softmax 并物化 $[B, H, S, S]$ score) 相对 TileLang 慢 11 至 13 倍，印证了 IO 感知分块在长序列下的优势。Prefill 的算术强度 (AI ≈ 1820 FLOP/Byte, causal, 序列长度 4096) 远高于屋脊点 (≈ 156 FLOP/Byte),

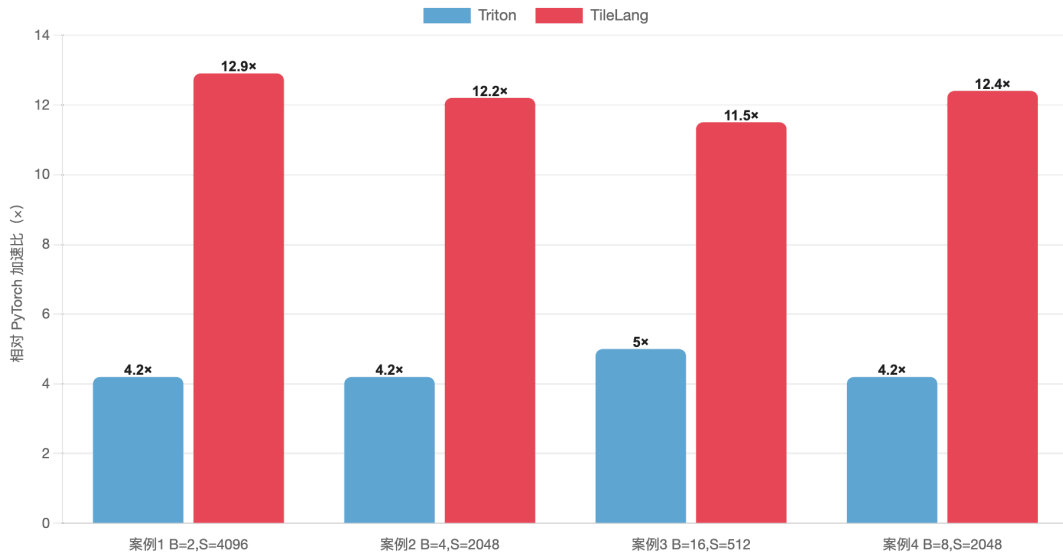


图 4-3 GQA Prefill 相对 PyTorch 加速比

属计算受限，瓶颈在 SM 占用与 Tensor Core 利用而非 HBM；序列长度 4096 时并行度已足以吃满 Tensor Core 流水。

4.2.4 GQA Decode 微基准

4.2.4.1 结果

表 4-6 为单步 Decode ($S_q=1$) 延迟中位数 (ms)；内核采用 Split-K, kv_len 见表中 kv 列。

表 4-6 GQA Decode 微基准延迟

案例	B	kv	PyTorch (ms)	Triton (ms)	TileLang (ms)
1	4	8192	6.81	0.343	0.155
2	8	4096	6.88	0.346	0.156
3	8	8192	13.53	0.645	0.229
4	16	4096	13.70	0.653	0.241

4.2.4.2 分析

Decode 场景下 TileLang Split-K 全面优于 Triton，四个测试用例的加速比为 $2.22\times$ 、 $2.22\times$ 、 $2.81\times$ 、 $2.71\times$ （见图 4-7）。加速最显著的是案例 3 ($B=8$, kv=8192)，达 $2.81\times$ ——长 KV 序列下 Split-K 的跨 SM 并行度优势被充分放大。

就案例 3 ($B=8$, kv=8192) 而言，HBM 流量约 268 MB，2.0 TB/s 对应时间下界约 0.134 ms，TileLang 实测 0.229 ms，带宽利用率约 59%；计入 Split-K 额

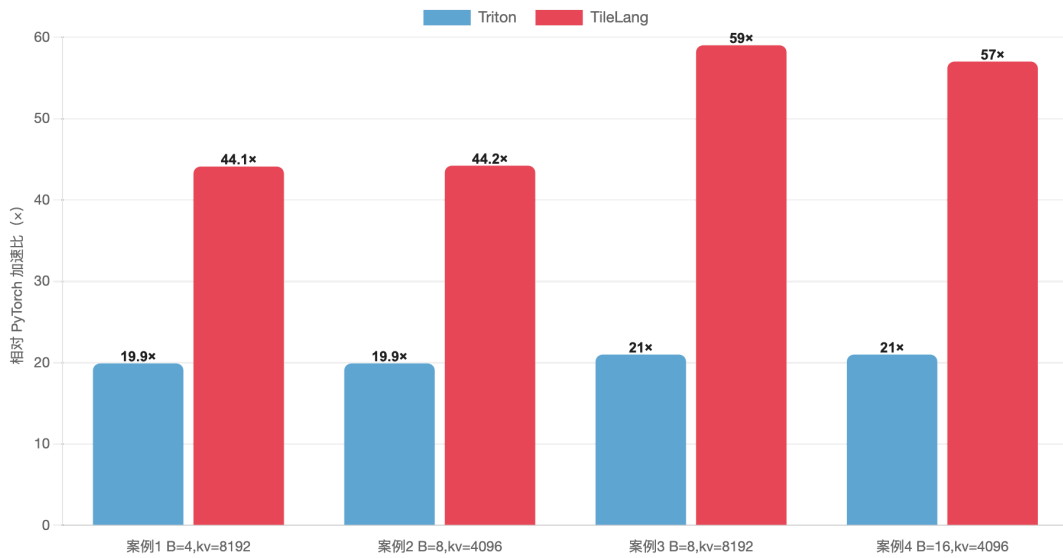


图 4-4 GQA Decode 相对 PyTorch 加速比

外归约后，该水平已属较高。

4.2.5 SwiGLU 微基准

4.2.5.1 结果

表 4-7 为融合 SwiGLU 延迟中位数 (ms)，中间维 $N=d_{\text{fm}}=27648$ ， $M=B \cdot S$ 由表 4-2 派生。

表 4-7 SwiGLU 微基准延迟

案例	M	PyTorch (ms)	Triton (ms)	TileLang (ms)
1	8192	1.305	0.824	0.829
2	8192	1.305	0.823	0.828
3	8192	1.306	0.850	0.833
4	16384	2.593	1.592	1.594

4.2.5.2 分析

TileLang 与 Triton 的融合实现相对 PyTorch 未融合 $F.\text{silu}(\text{gate}) * \text{up}$ 分别约 $1.65\times$ 与 $1.59\times$ ，与账面相符：未融合需 5 次 HBM 访问（4 读 + 2 写），融合仅 3 次（2 读 + 1 写），流量比 5:3，理论上界约 $1.67\times$ ，实测已逼近。两 DSL 彼此差距在 $0.99\text{--}1.02\times$ ，属同一带宽天花板下的噪声；SwiGLU 为纯逐元素 (elementwise) 融合，算术强度仅 $\approx 0.83 \text{ FLOP/Byte}$ （表 2-2），远低于屋脊点，性能几乎完全由 HBM 决定。不同于 RMSNorm 的两阶段归约或 GQA 的分块矩阵乘，SwiGLU 无

跨元素复用，只需各读 gate、up 并写回一次，编译器可榨取的余量有限。融合逐元素算子的天花板因而由 HBM 带宽锁定，TileLang 与 Triton 接近符合这一图景。

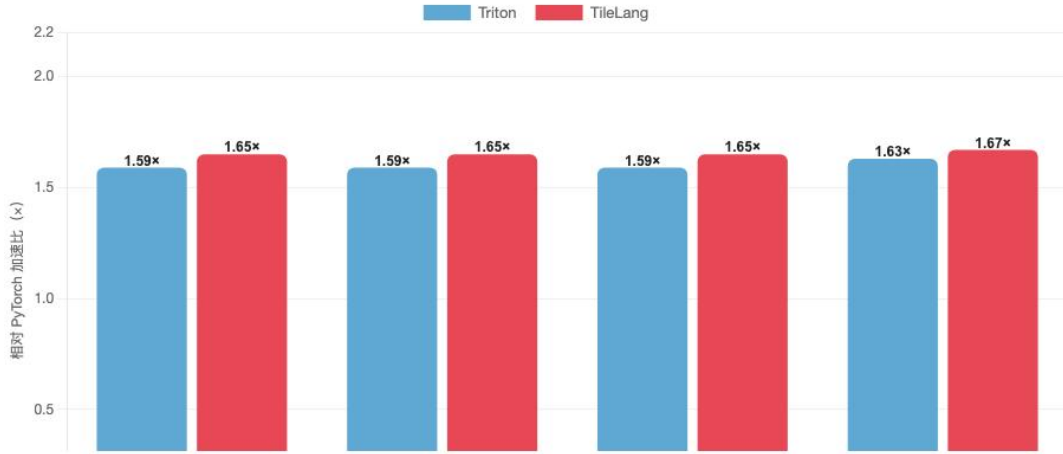


图 4-5 SwiGLU 相对 PyTorch 加速比

4.2.6 W4A16 GEMV 微基准

4.2.6.1 结果

测试取 Qwen3-32B 七类线性层实际权重形状， $M \in \{1, 4, 16\}$ 覆盖单请求到小批量 Decode 负载；group_size=128，逐组非对称 scale 与零点。表 4-8 为各层延迟中位数（ms），形状参数见表中 $N \times K$ 。

4.2.6.2 分析

整体上可归纳三点。Gate/Up/Down (25600×5120) 在 $M=1$ 时相对 PyTorch 约 $32\times$ ，LOP3 流水与 INT4 压缩是主因；K/V proj (640×5120) 仅 $4.7\times-4.9\times$ ，因矩阵小、启动开销占比高。TileLang 相对 Triton 的优势随 M 增大而拉开： $M=1$ 时大层约 $6\times$ ， $M=16$ 时约 $7\times$ ，且大层 Triton 可出现 $t_{Tri} > t_{Py}$ ，TileLang 仍保持 $t_{TL} \ll t_{Py}$ 。21 组形状上 TileLang 对 Triton 的加速分布在 $2.0-7.7\times$ （表 4-9）。

4.2.7 算子综合对比

表 4-9 汇总六类算子加速比区间（前五类取微基准四 case，W4A16 取 21 组形状），图 4-7 逐案例展示 t_{Tri}/t_{TL} （W4A16 仅三柱，对应 $M=1, 4, 16$ 七层均值）。

表 4-8 W4A16 GEMV 微基准延迟

M	层	$N \times K$	PyTorch (ms)	Triton (ms)	TileLang (ms)
1	Q proj	5120×5120	0.463	0.106	0.040
	K proj	640×5120	0.130	0.055	0.028
	V proj	640×5120	0.164	0.067	0.034
	O proj	5120×5120	0.465	0.118	0.034
	Gate proj	25600×5120	2.074	0.392	0.066
	Up proj	25600×5120	1.992	0.336	0.061
	Down proj	5120×25600	2.123	0.398	0.065
4	Q proj	5120×5120	0.463	0.280	0.051
	K proj	640×5120	0.175	0.089	0.037
	V proj	640×5120	0.173	0.088	0.034
	O proj	5120×5120	0.433	0.245	0.054
	Gate proj	25600×5120	1.988	1.317	0.174
	Up proj	25600×5120	1.985	1.090	0.161
	Down proj	5120×25600	1.991	1.332	0.174
16	Q proj	5120×5120	0.464	0.978	0.134
	K proj	640×5120	0.173	0.175	0.042
	V proj	640×5120	0.173	0.175	0.041
	O proj	5120×5120	0.433	0.810	0.117
	Gate proj	25600×5120	1.988	4.070	0.613
	Up proj	25600×5120	1.985	4.059	0.594
	Down proj	5120×25600	1.996	4.046	0.601

表中数据揭示三类结构性规律。

第一，片上复用收益显著。依赖片上数据复用的算子（RMSNorm 两阶段归约、GQA Prefill/Decode 分块矩阵乘）在 TileLang 下相对 Triton 可达 $1.20\text{--}3.08\times$ ，LayoutInference 与软件流水是主因。

第二，纯逐元素算子收益受限。纯逐元素（elementwise）的 SwiGLU 被 HBM 带宽封顶，TileLang 与 Triton 仅 $0.99\text{--}1.02\times$ 差距。

表 4-9 各算子加速比汇总

算子	TL 对 Tri	Tri 对 Py	TL 对 Py
RMSNorm	$1.20\text{--}1.37\times$	$6.9\text{--}7.2\times$	$8.6\text{--}9.8\times$
RoPE	$1.42\text{--}1.46\times$	$9.1\text{--}10.2\times$	$13.2\text{--}14.5\times$
GQA Prefill	$2.30\text{--}3.08\times$	$4.2\text{--}5.0\times$	$11.5\text{--}12.9\times$
GQA Decode	$2.22\text{--}2.81\times$	$19.9\text{--}21.0\times$	$44.0\text{--}59.0\times$
SwiGLU	$0.99\text{--}1.02\times$	$1.54\text{--}1.63\times$	$1.57\text{--}1.63\times$
W4A16 GEMV	$2.0\text{--}7.7\times$	$0.5\text{--}5.9\times$	$3.2\text{--}32.7\times$

第三，W4A16 GEMV 优势最大。W4A16 GEMV 差距最大（ $2.0\text{--}7.7\times$ ），因 LOP3 流水把 INT4 加载与反量化重叠，Triton 在大 M 下退化而 TileLang 仍占优。

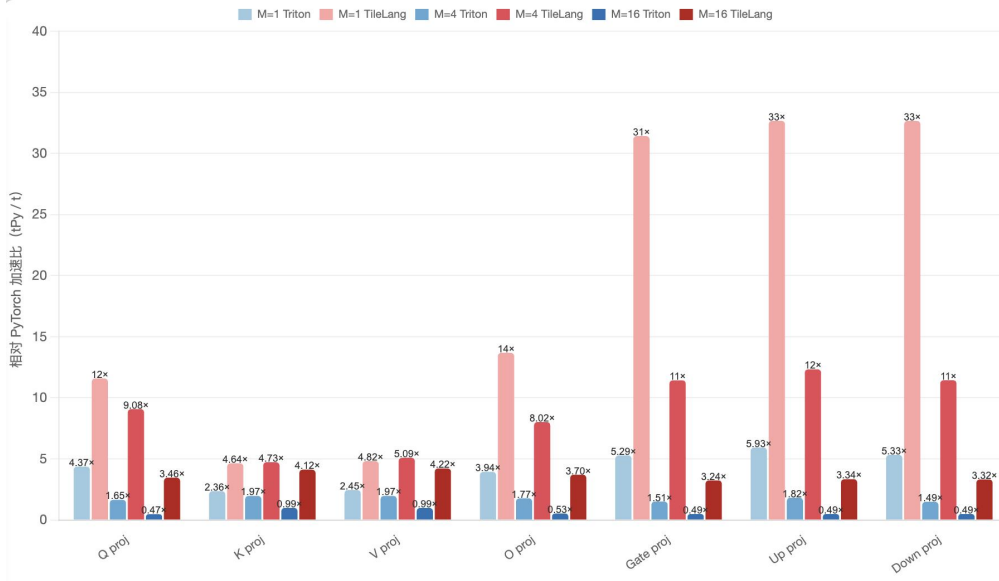


图 4-6 W4A16 GEMV 相对 PyTorch 加速比

图 4-8 以“有效性能 / 理论上界”量化六类算子三后端的硬件资源利用率。

第一，带宽受限算子口径。对 RMSNorm、RoPE、GQA Decode、SwiGLU 等带宽受限算子，取理论访存时间 $t_{bd} = V/B_{peak}$ ，其中 V 为第二章融合实现的最小 HBM 流量账， $B_{peak} = 2.0 \text{ TB/s}$ ；带宽利用率定义为 $\tilde{\eta}_{mem} = \min(100\%, t_{bd}/t \times 100\%)$ ，并对四组测试形状取算术平均。

第二，计算受限算子口径。GQA Prefill 属计算受限，用有效算力 $P_{eff} = F/t$ ，其中 $F = 2BH_qS^2D$ (causal)，计算利用率定义为 $\eta_{comp} = \min(100\%, P_{eff}/P_{peak} \times 100\%)$ ， $P_{peak} = 312 \text{ TFLOPS}$ ，同样对四个 case 平均。

第三，W4A16 口径。W4A16 GEMV 为流式访存场景， $M=1$ 时内存请求密度极低，由实测反算可得可持续有效带宽约为峰值的 24%，故取 $B_{eff} = 0.48 \text{ TB/s}$ 作为该算子的参考上界；字节账为 $V = NK/2 + 2MK + 2MN + 8NK/128$ (INT4 权重、FP16 激活读写、逐组 scale/零点)，同样使用 $\tilde{\eta}_{mem}$ 并对 $M \in \{1, 4, 16\}$ 共 21 组求均值。该口径仅用于三后端横向对照，不与 FP16 五类混比绝对值。

从图 4-8 可见，带宽受限侧 TileLang 普遍贴近硬件上界：RMSNorm 约 95.3%、GQA Decode 约 93.2%，两阶段 inv_std 复用与 Split-K 分块消弭了主要访存瓶颈。SwiGLU 的 TileLang (82.7%) 与 Triton (82.6%) 几乎触顶同一带宽天花板，纯逐元素算子缺乏片上复用，三后端差异主要来自 HBM 利用率而非调度技巧，与第二章 Roofline 判断一致。

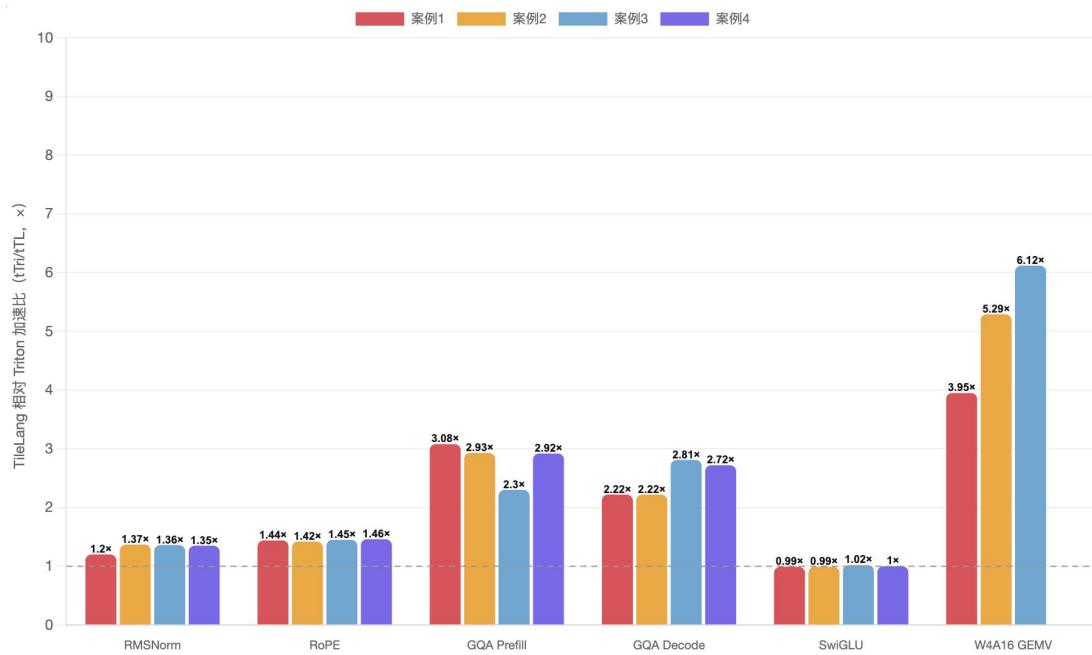


图 4-7 各算子 TileLang 相对 Triton 加速比

计算受限的 GQA Prefill 三后端绝对利用率都不高，TileLang 约 41.5% 相对 Triton 14.6% 提升约 2.8 倍，得益于 LayoutInference 消除 shared memory bank conflict 与 T.Pipelined；PyTorch 仅 3.4%，因物化序列长度平方的 score 矩阵引入大量 HBM 往返。

W4A16 上 TileLang 约 50.2% 相对 Triton 13.6% 提升约 3.7 倍，LOP3 流水把加载与解码重叠；PyTorch 5.9%，eager 路径未针对 INT4 访存优化。

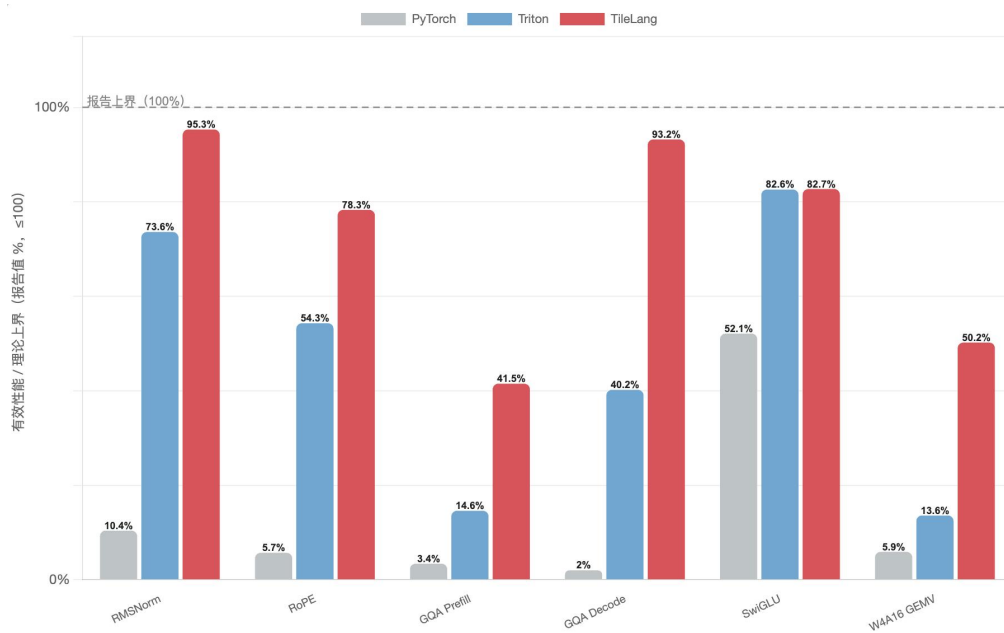


图 4-8 六类算子相对理论上界利用率

4.3 端到端系统评估

4.3.1 结果

在完整 Qwen3-32B 推理链路上对比三后端的端到端指标。测试场景为多 Agent 代码分析：静态批处理， $B \in \{4, 8\}$ ， $\text{prompt} \in \{512, 1024, 2048\}$ tokens， $\text{gen_len}=128$ ；重复 3 次取中位数。

表 4-10 端到端系统评估

Batch	Prompt	生成吞吐 (tok/s)			TL/Tri	墙上 TL (s)	墙上 Tri (s)
		PyTorch	Triton	TileLang			
4	512	10.7	32.9	49.3	1.50×	11.0	16.2
	1024	10.5	31.1	47.2	1.51×	12.2	17.7
	2048	10.3	29.1	46.9	1.61×	13.5	20.2
8	512	21.2	40.2	81.1	2.01×	14.4	27.2
	1024	20.5	37.7	71.5	1.90×	17.1	29.9
	2048	19.5	34.5	60.5	1.76×	22.8	35.6

图 4-9 与表 4-10 同源：W4A16 权重路径， $B \in \{4, 8\}$ ， $\text{prompt} \in \{512, 1024, 2048\}$ token， $\text{gen_len}=128$ ，greedy decoding，A100；纵轴示生成吞吐随批量与 prompt 长度变化（静态批、三次重复取中位数）。

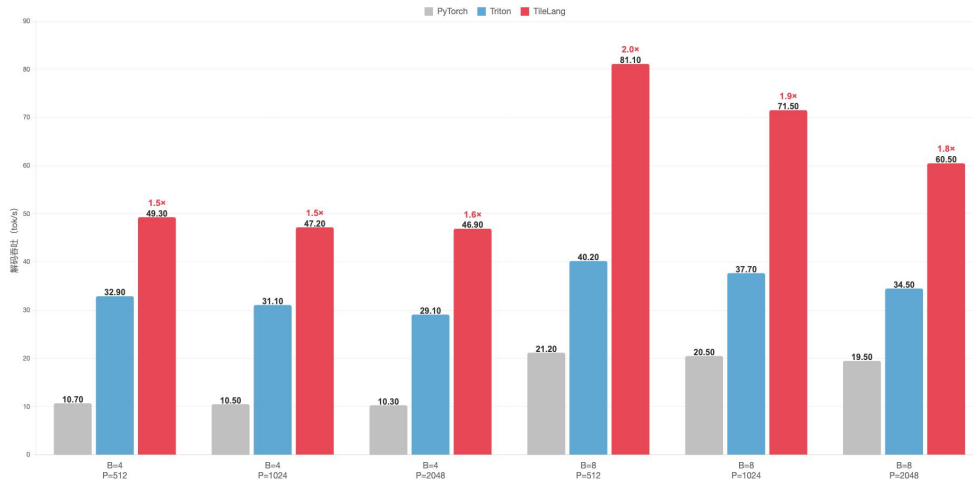


图 4-9 端到端 Decode 吞吐

4.3.2 分析

批量为 4 时 TileLang 达 1.50 至 1.61 倍（49.3 对 32.9 tok/s， $\text{prompt}=512$ ），批量为 8 时进一步扩大至 1.76 至 2.01 倍（81.1 对 40.2 tok/s， $\text{prompt}=512$ ；71.5 对

37.7 tok/s, prompt=1024)。prompt 越长, KV Cache 读取量越大, 访存压力随之上升, 相同批量下 Triton 与 TileLang 的绝对吞吐均下降, 但 TileLang 下降更平缓, 加速比因此随 prompt 增大而略升 (批量 4 时从 1.50 升至 1.61 倍, 批量 8 时从 2.01 降至 1.76 倍; 后者因 prompt=512 时带宽利用率已接近上限, 此处加速比最高)。墙上时间列与吞吐趋势一致: 批量为 8、prompt=1024 时 Triton 耗时 29.9 s, TileLang 仅需 17.1 s, 缩短 43%; prompt=2048 时缩短 36%。加速根本来源是 LOP3 流水线在大批量下充分利用有效带宽, 将 Gate/Up/Down 投影层 (各约 71 MB INT4) 的 HBM 读取与反量化计算完全重叠, 而 Triton 在大批量下访存效率下降, 差距在全链路中进一步放大。

4.4 消融实验

本节通过四组消融实验, 量化各关键设计决策对性能的独立贡献。

4.4.1 软件流水对 GQA Prefill 的贡献

TileLang 通过 T.Pipelined 将 K/V tile 的 HBM 读取与 QK^T 计算重叠 (多阶段异步预取)。表 4-11 在 $B=2, S=4096$ (causal) 下对比不同 num_stages 的内核延迟与 SM 吞吐, 量化异步流水带来的收益。此处数据来自 ncu 单次采集 (固定 block_M=128, block_N=64, threads=256), 与微基准 (表 4-5) 的 AutoTune 全参数搜索结果 (3.59 ms) 存在差异, 两者测量条件不同。

表 4-11 GQA Prefill 软件流水消融

实现	延迟 (ms)	SM 吞吐占峰值 (%)
num_stages=1 (无流水重叠)	4.921	45.7
num_stages=2 (T.Pipelined)	4.424	50.8

开启 2 阶段流水相比单阶段性能提升 1.11 倍, SM 吞吐由 45.7% 提升至 50.8%, 与第三章 ncu 分析中 Long Scoreboard 停顿显著下降的现象一致: 流水化使 K/V 预取与当前块上的 T.gemm 得以重叠, 减少了 warp 在 HBM 上的空转时间。

4.4.2 Split-K 分割度对 GQA Decode 的影响

Decode 阶段 ($S_q=1$) 是典型的访存受限 (memory-bound) 算子, 增大 num_split 可以让更多 SM 并行处理不同的 KV chunk, 但过度分割会引入额

外的跨 CTA LSE 归约开销。表 4-12 以 $B=16$, $kv=4096$ 为例，固定其余参数 ($block_N=128$, $block_H=64$, $threads=128$)，仅变动 num_split 。

表 4-12 GQA Decode Split-K 消融

num_split	延迟 (ms)	有效算力 (TFLOPS)
1	0.266	8.07
2	0.226	9.40
4	0.217	9.85
8	0.238	9.04
16	0.277	7.74

$num_split=4$ 为最优，相比不分割快 $1.23\times$ ($8.07\rightarrow9.85$ TFLOPS)。继续增大到 8、16 时性能下降，split 粒度过细导致 SM 间 LSE 归约的同步开销超过并行收益。这一“倒 U 型”特征说明存在最优并行粒度，AutoTune 通过搜索 480 个候选自动定位这一平衡点。

4.4.3 AutoTune 搜索的必要性

RMSNorm 的搜索空间由 ($blk_m, blk_k, threads$) 三元组构成，共 256 个候选。表 4-13 在固定形状 $M=8192, N=5120$ 下，固定 $blk_k=1024, threads=256$ ，仅改变 blk_m ，以在缩减参数空间内说明该搜索的非单调性。

表 4-13 RMSNorm AutoTune 收益示意

配置	blk_m	blk_k	threads	延迟 (ms)
锚点 A (偏小)	1	1024	256	0.167
AutoTune 最优	2	1024	256	0.137
锚点 B (偏大)	4	1024	256	0.267

搜索空间并非单调：锚点 A 的 $blk_m=1$ 每个 CTA 仅处理 1 行，线程资源利用不足；锚点 B 的 $blk_m=4$ 每个 CTA 处理 4 行，寄存器与共享内存占用激增，导致 SM 占用率 (occupancy) 下降，反而比 A 更慢 (0.267 ms vs. 0.167 ms)。AutoTune 搜索到的 $blk_m=2$ 恰处于两者之间，延迟 0.137 ms，分别较锚点 A、B 快 $1.22\times$ 和 $1.95\times$ 。这说明最优切片策略受访存带宽、寄存器占用与 SM 并行度的共同约束，人工凭直觉选取极端配置均难以命中，AutoTune 对全候选集的系统性搜索是找到该平衡点的必要手段。GQA Prefill 等矩阵乘类算子同样在百至数百量级的候选空间上采用同一套 AutoTune 流程；其在微基准中的峰值有效算

力（表 4-5 案例 1，153 TFLOPS）亦得益于此。

4.4.4 DSL 抽象层次退化链

表 4-14 以 RMSNorm ($M=8192, N=5120$) 为例，展示从 TileLang 到 PyTorch 原生实现的逐步退化。

表 4-14 RMSNorm 实现路径退化链

实现	延迟 (ms)	相对 TileLang
TileLang (融合, autotune)	0.128	1.0×
Triton (融合, 手写内核)	0.174	1.36× 慢
PyTorch F.rms_norm	1.151	9.0× 慢
PyTorch naive (两步分离)	1.336	10.4× 慢

TileLang 比 Triton 快约 36% (0.128 与 0.174 ms)，得益于 AutoTune 确定的更优 (`blk_m, blk_k, threads`) 与 TileLang 编译器的布局推导、流水插入等 Pass 的联合作用。两者相对 PyTorch eager 均有数量级提升。F.rms_norm 的 aten 实现将 RMS 计算与权重乘法分为两个独立 kernel，产生额外的 HBM 往返；PyTorch naive 实现更要额外物化平方和张量，访存次数进一步增加。这一退化链表明：在融合归约算子上，TileLang 路线能够以更简洁的内核描述获得优于手写 Triton 的稳态性能，同时相对框架默认路径保持显著优势。

4.5 交互展现层功能验证

本节对交互展现层两个功能模块进行实测，展示其面向各自使用方的实际功能效果。

4.5.1 多 Agent 并发推理模块实测

多 Agent 并发推理模块面向普通用户，支持用户自行填写代码上下文与问题描述，并为每个 Agent 独立指定分析任务。模块的核心设计目标是验证系统在多路并发负载下能否维持低延迟、高吞吐的推理性能，同时保证各路输出的独立性与正确性。

实测中配置 4 路并发并指定 TileLang 后端，四个 Agent 分别承担设计思路分析、性能瓶颈分析、量化误差分析与参数调优建议四项任务，共享同一份代码上下文，各自携带独立的系统提示词与任务指令。提交后，服务端并行启动四

路推理请求，各路输出通过流式接口实时推送至前端展示区，四列结果并排逐 token 显示，用户可在任意一路生成完成之前就开始阅读已到达的内容，无需等待全部完成。实测结果如图 4-10 所示，四路在相近时刻均完成生成，未出现某路因资源争用而显著滞后的情况，验证了多 Agent 场景下并发调度的均衡性。

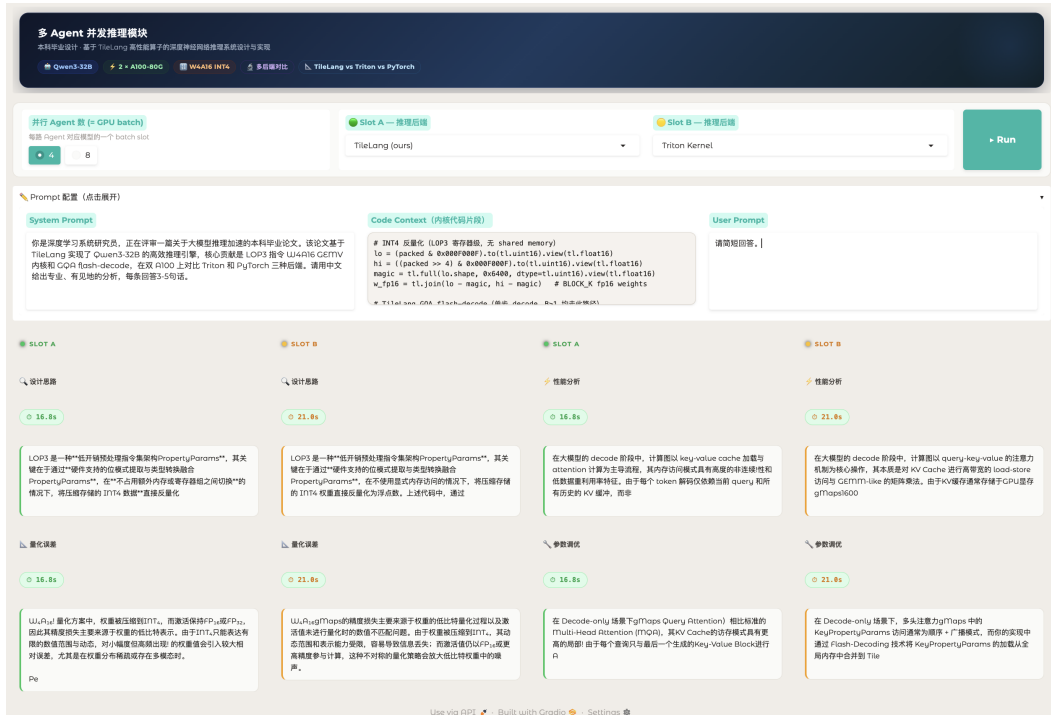


图 4-10 多 Agent 并发推理模块实测

4.5.2 后端分析监控模块实测

后端分析监控模块面向服务提供商的管理人员，其目标是在不要求用户深入代码的前提下，提供对推理服务运行状况与性能水平的直观感知能力。模块分为日志面板与性能面板两个视图：日志面板实时滚动展示服务端标准输出，管理人员可第一时间发现异常报错或超时警告；性能面板以数值卡片形式呈现当前推理任务的墙上时间（WALL）与吞吐速率（TOK/S），刷新闻隔与推理完成事件同步，不依赖轮询。

实测流程为：在同一提示词与批次配置下，先后以 TileLang (ours) 与 Triton Kernel 作为推理后端各运行一次，模块自动采集并在性能面板上并列展示两次结果。如图 4-11 所示，TileLang 侧墙上时间 13.44 s、吞吐 46.5 tok/s，Triton 侧墙上时间 19.26 s、吞吐 32.0 tok/s，TileLang 吞吐提升约 1.45 倍、墙上时间缩短约 30%。管理人员无需执行任何基准脚本，仅凭面板数字即可直接量化后端切换带

来的性能收益，验证了监控模块在实际运维场景中的可用性。

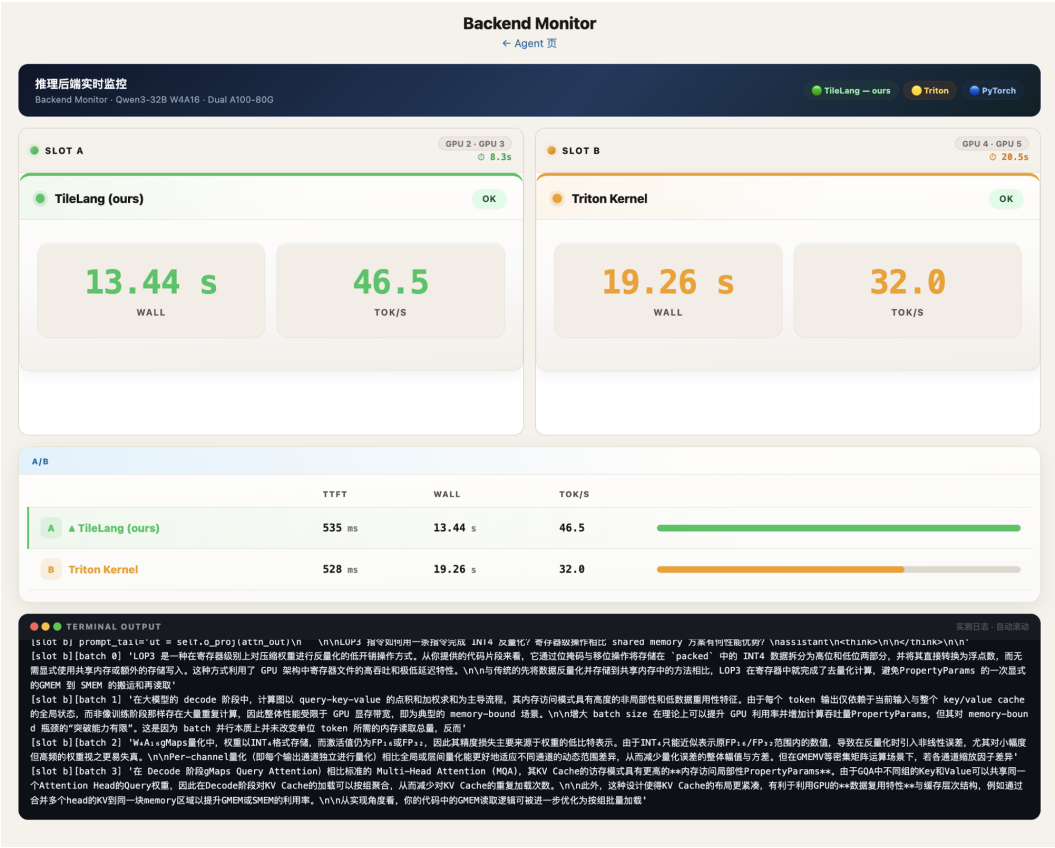


图 4-11 后端分析监控模块实测

4.6 本章小结

本章对六类算子进行了微基准测试、算子级综合对比、端到端吞吐评估与消融实验，定量分析了各项设计的性能贡献。

5 总结与展望

5.1 主要工作总结

本文基于国产 DSL TileLang 构建了从算子内核到交互演示的完整推理原型。针对 RMSNorm、RoPE、GQA Prefill、GQA Decode、SwiGLU 与 W4A16 GEMV，以 AutoTune 自动确定 tile；除 SwiGLU 因触顶带宽与 Triton 接近外，其余五类相对 Triton 均有显著加速，端到端 W4A16 路径在批大小为 4 时性能提升约 1.5 至 1.6 倍、批大小为 8 时性能提升约 1.8 至 2.0 倍，墙上时间缩短 30% 至 50%。配套实现多 Agent 场景下的交互展现层，支持多路并发推理与多后端性能对比，便于可视化验证算子收益。评估上，本文在微基准、端到端与消融三个层次分层度量，并结合 Nsight Compute 等指标完成从现象到机制的归因。

5.2 展望

5.2.1 算子层面

在算子层，W4A16 目前对所有权重统一 4-bit 分组量化；更可取的方向是依据层间 Hessian 或激活长尾自适应保留高精度，但这要求 GPTQ、AWQ 等与 TileLang 内核深度耦合，DSL 侧尚缺成熟量化感知编译支持。

另一方面，算子仍依赖人工编写 TileLang，借助 KernelBench^[29] 等框架由大模型或搜索自动生成内核有望降门槛，但正确性校验与搜索成本仍是开放问题。

5.2.2 系统层面

系统上，本文原型采用静态批处理，尚未纳入连续批处理与分页注意力 (PagedAttention)^[30]；若要将 TileLang 作为可插拔内核嵌入 vLLM、SGLang^[31]，须补齐接口适配与多卡张量并行下的 All-Reduce 融合等工程环节。

投机解码^[32] 与混合专家 (MoE) 推理亦是前沿方向，将本文方法迁移到这些负载值得后续探索。

5.2.3 生态层面

生态上，TileLang 的解耦设计理论上可映射到华为 Ascend、海光 DCU、寒武纪 MLU 等国产平台，但各平台存储层次与工具链差异大，需逐平台重写与验

证。

演示侧 Agent 逻辑目前为内部定制，后续若将后端以 OpenAI-compatible API 暴露并接入 OpenClaw 等真实框架，可在实际 Agent 负载下进一步检验算子收益。

5.3 本章小结

本章总结了六类算子的优化成果与系统实现，并展望了算子融合、更低位宽量化及多框架集成等后续方向。TileLang 在归约与矩阵乘类算子上展现出明显工程收益，所建立的评估方法论与交互验证框架为后续研究提供了可复用基础。

参考文献

- [1] Anthropic. Claude code: An agentic coding tool. <https://docs.anthropic.com/en/docs/claude-code>, 2025.
- [2] Peter Steinberger et al. Openclaw: Your own personal ai assistant. <https://github.com/openclaw/openclaw>, 2025.
- [3] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024.
- [4] Md. Monzurul Amin Ifath and Israat Haque. Characterizing performance–energy trade-offs of large language models in multi-request workflows. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 10(1):1–26, March 2026.
- [5] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, pages 16344–16359, 2022.
- [6] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.
- [7] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, pages 68658–68685, 2024.
- [8] Philippe Tillet, Hsiang-Tsung Kung, and David D. Cox. Triton: an intermediate

- language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, MAPL@PLDI 2019, Phoenix, AZ, USA, June 22, 2019*, pages 10–19. ACM, 2019.
- [9] Lei Wang, Lingxiao Ma, Shijie Cao, Quanlu Zhang, Jilong Xue, Zhi Yang, and Mao Yang. Tilelang: A composable tiled programming model for AI systems. *arXiv preprint arXiv:2504.17577*, 2025.
- [10] NVIDIA. CUTLASS: CUDA templates for linear algebra subroutines. <https://github.com/NVIDIA/cutlass>, 2017.
- [11] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: an automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018*, pages 578–594. USENIX Association, 2018.
- [12] Siyuan Feng, Bohan Hou, Hongyi Jin, Wuwei Lin, Junru Shao, Ruihang Lai, Zihao Ye, Lianmin Zheng, Cody Hao Yu, Yong Yu, and Tianqi Chen. TensorIR: An abstraction for automatic tensorized program optimization. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, pages 804–817. ACM, 2023.
- [13] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman P. Amarasinghe. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13, Seattle, WA, USA, June 16-19, 2013*, pages 519–530. ACM, 2013.
- [14] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques A. Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and

- Oleksandr Zinenko. MLIR: scaling compiler infrastructure for domain specific computation. In *IEEE/ACM International Symposium on Code Generation and Optimization, CGO 2021, Seoul, South Korea, February 27 - March 3, 2021*, pages 2–14. IEEE, 2021.
- [15] Benjamin F. Spector, Simran Arora, Aaryan Singhal, Abhimanyu Rajeshkumar, Tri Dao, and Christopher Ré. Thunderkittens: Simple, fast, and adorable AI kernels. *arXiv preprint arXiv:2410.20399*, 2024.
- [16] Yining Shi, Zhi Yang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Ziming Miao, Yuxiao Guo, Fan Yang, and Lidong Zhou. Welder: Scheduling deep learning memory access via tile-graph. In *17th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2023, Boston, MA, USA, July 10-12, 2023*, pages 701–718. USENIX Association, 2023.
- [17] Yu Cheng, Lei Wang, Yining Shi, Yuqing Xia, Lingxiao Ma, Jilong Xue, Yang Wang, Zhiwen Mo, Feiyang Chen, Fan Yang, Mao Yang, and Zhi Yang. Pipethreader: Software-defined pipelining for efficient DNN execution. In *19th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2025, Boston, MA, USA, July 7-9, 2025*, pages 767–783. USENIX Association, 2025.
- [18] Samuel Williams, Andrew Waterman, and David A. Patterson. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM*, 52(4):65–76, 2009.
- [19] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [20] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 4895–4901. Association for Computational Linguistics, 2023.
- [21] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In

- Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 12360–12371, 2019.
- [22] Jianlin Su, Murtadha H. M. Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [23] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- [24] Guangxuan Xiao, Ji Lin, Mickaël Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, Proceedings of Machine Learning Research, pages 38087–38099. PMLR, 2023.
- [25] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024*. mlsys.org, 2024.
- [26] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. OPTQ: accurate quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- [27] Elias Frantar, Roberto L. Castro, Jiale Chen, Torsten Hoefler, and Dan Alistarh. MARLIN: mixed-precision auto-regressive parallel inference on large language models. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, PPOPP 2025, Las Vegas, NV, USA, March 1-5, 2025*, pages 239–251. ACM, 2025.
- [28] Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. Flash-

- decoding for long-context inference. <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>, 2023.
- [29] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR / OpenReview.net, 2025.
- [30] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, pages 611–626. ACM, 2023.
- [31] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, pages 62557–62583, 2024.
- [32] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024–1 May 2024*, pages 932–949. ACM, 2024.